



Instituto Federal Catarinense
Curso de Bacharelado em Ciência da Computação
Campus Blumenau

LEONARDO DE SOUZA FIAMONCINI

**APACHE KAFKA E RABBITMQ: UMA ANÁLISE COMPARATIVA ENTRE
SISTEMAS DE MENSAGERIA EM APLICAÇÕES DE DIFERENTES PORTES**

BLUMENAU

2025

LEONARDO DE SOUZA FIAMONCINI

**APACHE KAFKA E RABBITMQ: UMA ANÁLISE COMPARATIVA ENTRE
SISTEMAS DE MENSAGERIA EM APLICAÇÕES DE DIFERENTES PORTES**

Trabalho de Conclusão de Curso apresentado ao
Curso de Ciência da Computação do Instituto
Federal Catarinense – *Campus* Blumenau – como
requisito parcial para a obtenção do título de
Bacharel em Ciência da Computação.

Orientador: Prof. Dr. Hylson Vescovi Netto.

BLUMENAU

2025

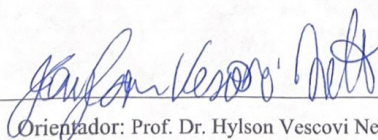
LEONARDO DE SOUZA FIAMONCINI

APACHE KAFKA E RABBITMQ: UMA ANÁLISE COMPARATIVA ENTRE
SISTEMAS DE MENSAGERIA EM APLICAÇÕES DE DIFERENTES PORTES

Trabalho de Conclusão de Curso apresentado ao
Curso de Ciência da Computação do Instituto
Federal Catarinense – *Campus* Blumenau – como
requisito parcial para a obtenção do título de
Bacharel em Ciência da Computação.

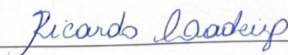
Orientador: Prof. Dr. Hylson Vescovi Netto.

Aprovado em: 15/12/2025.



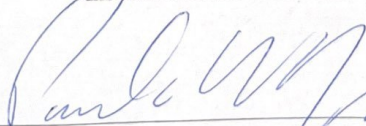
Orientador: Prof. Dr. Hylson Vescovi Netto

Instituto Federal Catarinense



Membro: Prof. M.e Ricardo de la Rocha Ladeira

Instituto Federal Catarinense



Membro: Prof. *Postdoc* Dr. Paulo Cesar Rodacki Gomes

Instituto Federal Catarinense

AGRADECIMENTOS

À Isabella, meu amor, cujo carinho e afeto incondicionais me fortaleceram e me motivaram a nunca desistir dos meus objetivos.

À minha família, em especial aos meus pais, por serem o alicerce da minha formação e pelo apoio constante que moldou quem me tornei.

Ao Instituto Federal Catarinense, por desempenhar um papel essencial no início da minha trajetória na área de tecnologia e por contribuir para a minha formação.

A Deus, pela vida, pela presença constante e por me conceder forças ao longo de toda esta caminhada.

RESUMO

A onipresença de arquiteturas baseadas em microsserviços e a demanda por processamento de dados em tempo real exacerbaram desafios clássicos de sistemas distribuídos, notadamente o acoplamento temporal e a latência na comunicação interprocessual. Como solução, o uso de *message brokers* viabiliza a comunicação assíncrona, oferecendo persistência e desacoplamento entre produtores e consumidores. Sendo assim, este trabalho apresenta uma análise comparativa de desempenho entre o RabbitMQ (versão 4.1.1) e o Apache Kafka (versão 4.0), ambos configurados com seus respectivos mecanismos de consenso e alta disponibilidade mais recentes: *Quorum Queues* e protocolo KRaft, respectivamente. Além dessas duas tecnologias, incluiu-se uma linha de base (*baseline*) síncrona via HTTP como referência comparativa. O experimento submeteu cada arquitetura a cinco perfis de carga progressivos (de 100 a 1.000.000 de mensagens), variando também o tamanho da carga útil (*payload*) entre 100 bytes e 1.000 bytes. As métricas de avaliação concentraram-se na vazão e na latência de cauda (percentis 95 e 99), críticas para a garantia de SLA em sistemas de larga escala. Os resultados indicaram que, embora a comunicação síncrona apresente latência inferior em baixas cargas, ela sofre de severa limitação de vazão, estagnando sua capacidade de processamento. Já o RabbitMQ demonstrou o melhor equilíbrio entre latência e vazão em cargas intermediárias. O Apache Kafka, contudo, exibiu latência elevada em baixos volumes de tráfego de mensagens devido ao seu mecanismo de *batching*, mas superou todas as alternativas no cenário de maior escala (1.000.000 de mensagens), validando sua arquitetura orientada a *logs* para *Big Data*. O delineamento experimental priorizou a validade interna e a reprodutibilidade, com o código-fonte disponibilizado publicamente. Desse modo, os resultados obtidos fornecem evidências para subsidiar a seleção arquitetural do *middleware* de mensageria mais adequado, considerando o *trade-off* entre latência, consistência e volume de dados da aplicação.

Palavras-chave: Apache Kafka; alta disponibilidade; desempenho; *message brokers*; RabbitMQ; sistemas distribuídos.

ABSTRACT

The ubiquity of microservices-based architectures and the demand for real-time data processing have exacerbated classic challenges in distributed systems, notably temporal coupling and inter-process communication latency. The use of message brokers emerges as a solution, enabling asynchronous communication and offering persistence and decoupling between producers and consumers. This work presents a comparative performance analysis between RabbitMQ (version 4.1.1) and Apache Kafka (version 4.0), both configured with their respective latest consensus and high-availability mechanisms: Quorum Queues and the KRaft protocol. A synchronous HTTP baseline was also included as a comparative reference. The experiment subjected each architecture to five progressive load profiles (ranging from 100 to 1,000,000 messages), also varying the payload size between 100 bytes and 1,000 bytes. The evaluation metrics focused on throughput and tail latency (95 and 99 percentiles), which are critical for guaranteeing SLA in large-scale systems. The results indicated that while synchronous communication presented lower latency under light loads, it suffered from severe throughput limitation, stagnating its processing capacity. RabbitMQ demonstrated the best balance between latency and throughput under intermediate loads. Apache Kafka, however, exhibited elevated latency at low message traffic volumes due to its batching mechanism, but outperformed all alternatives in the largest-scale scenario (1,000,000 messages), validating its log-oriented architecture for Big Data processing. The experimental design prioritized internal validity and reproducibility, with the source code publicly available. Therefore, the results obtained aim to provide evidence to support the architectural selection of the most suitable messaging middleware, considering the trade-off between latency, consistency, and application data volume.

Keywords: Apache Kafka; distributed systems; high-availability; message brokers; performance; RabbitMQ.

LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama de camadas de um Sistema Distribuído.....	19
Figura 2 – Teorema CAP.....	21
Figura 3 – Fluxo de mensagens no RabbitMQ.....	24
Figura 4 – Fluxo de gravação no Apache Kafka.....	27
Figura 5 – Comparativo de Vazão para mensagens de 100 bytes.....	43
Figura 6 – Comparativo de Vazão para mensagens de 1.000 bytes.....	43
Figura 7 – Comparativo da Latência P95 para mensagens de 100 bytes.....	46
Figura 8 – Comparativo da Latência P95 para mensagens de 1.000 bytes.....	46
Figura 9 – Comparativo da Latência P99 para mensagens de 100 bytes.....	47
Figura 10 – Comparativo da Latência P99 para mensagens de 1.000 bytes.....	47

LISTA DE TABELAS

Tabela 1 – Resultados Experimentais para Mensagens de 100 bytes.....	40
Tabela 2 – Resultados Experimentais para Mensagens de 1.000 bytes.....	41

LISTA DE ABREVIATURAS E SIGLAS

AMQP	Advanced message queuing protocol
CAP	Consistency, availability, partition tolerance
CPU	Central processing unit
CSV	Comma-separated values
HTTP	Hypertext transfer protocol
JSON	Javascript object notation
JVM	Java virtual machine
KRAFT	Kafka raft metadata
MQTT	Message queuing telemetry transport
QOS	Quality of service
RAM	Random access memory
REST	Representational state transfer
RTT	Round trip time
SLA	Service level agreement
SSD	Solid state drive
UUID	Universally unique identifier
WSGI	Web server gateway interface

SUMÁRIO

1	INTRODUÇÃO	12
1.1	OBJETIVOS	13
1.1.1	Objetivo geral.....	13
1.1.2	Objetivos específicos.....	13
1.2	JUSTIFICATIVA	14
2	REVISÃO DE LITERATURA.....	16
2.1	SISTEMAS DISTRIBUÍDOS	16
2.1.1	Fundamentos e Aplicabilidade.....	16
2.1.2	Comunicação em Sistemas Distribuídos.....	17
2.1.2.1	<i>Comunicação Síncrona.....</i>	<i>17</i>
2.1.2.2	<i>Comunicação Assíncrona.....</i>	<i>18</i>
2.1.3	Arquitetura e Classificação por Escala	18
2.2	DESAFIOS EM SISTEMAS DISTRIBUÍDOS	20
2.2.1	Falhas e Tolerância.....	20
2.2.2	Disponibilidade e Sobrecarga.....	22
2.3	RABBITMQ	23
2.3.1	Conceito e Fundamentos.....	23
2.3.2	Arquitetura e Componentes	23
2.3.3	Aplicabilidade e Casos de Uso.....	25
2.4	APACHE KAFKA	26
2.4.1	Conceito e Fundamentos.....	26
2.4.2	Arquitetura e Componentes	27
2.4.3	Aplicabilidade e Casos de Uso.....	28
2.5	MÉTRICAS DE AVALIAÇÃO DE DESEMPENHO	29
2.5.1	Latência e Percentis.....	29
2.5.2	Vazão.....	30
3	METODOLOGIA.....	31
3.1	AMBIENTE EXPERIMENTAL	31
3.1.1	Especificações de <i>Hardware</i> e <i>Software</i>.....	32
3.1.2	Definição e Configuração dos <i>Brokers</i>.....	32
3.2	DELINEAMENTO DOS CENÁRIOS DE TESTE	33
3.3	INSTRUMENTAÇÃO E COLETA DE DADOS	34

3.3.1	Sincronização e Precisão Temporal.....	34
3.3.2	Estrutura de Dados e Persistência	35
3.3.3	Tratamento Estatístico e Percentis	36
3.4	PROCEDIMENTO DE EXECUÇÃO.....	35
4	TRABALHOS CORRELATOS.....	36
5	DESENVOLVIMENTO.....	40
5.1	CONSOLIDAÇÃO DOS DADOS OBTIDOS.....	40
5.2	ANÁLISE DE VAZÃO.....	42
5.2.1	O Teto de Processamento do Modelo Síncrono	44
5.2.2	A Eficiência do RabbitMQ em Cargas Intermediárias.....	44
5.2.3	O Ponto de Inflexão do Apache Kafka	45
5.3	ANÁLISE DE LATÊNCIA DA CAUDA (P95 E P99).....	45
5.3.1	O Custo da <i>Batching</i> no Apache Kafka.....	48
5.3.2	A Estabilidade das <i>Quorum Queues</i> no RabbitMQ.....	48
5.3.3	Degradação em Escala Massiva.....	48
5.4	SÍNTESE COMPARATIVA E DIRETRIZES ARQUITETURAIS.....	49
6	CONCLUSÃO.....	51
	REFERÊNCIAS.....	54

1 INTRODUÇÃO

A popularização de microserviços, computação em nuvem e *Internet* das Coisas multiplicou significativamente o fluxo de eventos que transitam entre processos distribuídos, expondo gargalos de comunicação. Em tais contextos, a transmissão de dados por chamadas síncronas eleva o acoplamento entre serviços e aumenta o risco de indisponibilidade do sistema. Como forma de mitigar esses problemas, surgem os *message brokers* – *softwares* responsáveis por receber, armazenar temporariamente e encaminhar mensagens de forma assíncrona – que desacoplam emissores de receptores, aumentando a resiliência do sistema (EUGSTER *et al.*, 2003).

Dentre as soluções de mercado, o RabbitMQ e o Apache Kafka consolidaram-se como pilares na engenharia de dados moderna. Pesquisas recentes da indústria, como o *Stack Overflow Developer Survey* (2024), continuam a posicionar ambas as tecnologias entre as ferramentas de mensageria mais utilizadas globalmente, validando sua relevância contínua mais de uma década após seus lançamentos iniciais. O RabbitMQ, lançado em 2007, implementa o protocolo AMQP (*Advanced Message Queuing Protocol*) e enfatiza roteamento flexível e garantias de entrega individuais. O Apache Kafka, originalmente desenvolvido pelo LinkedIn e tornado *open source* em 2011, propôs uma mudança de paradigma ao adotar um modelo de *log* distribuído particionado, priorizando alta vazão e o reprocessamento de dados via deslocamento de leitura (*offset*). Estudos comparativos prévios indicam, tradicionalmente, que o Kafka atinge maior vazão, enquanto o RabbitMQ oferece menor latência em cargas moderadas (SOUZA, 2020).

Estas conclusões, contudo, baseiam-se em versões anteriores a mudanças arquiteturais recentes e significativas. O Apache Kafka, na versão 4.0, eliminou a dependência do ZooKeeper através do protocolo KRaft (*Kafka Raft Metadata*), simplificando a gestão de metadados e a infraestrutura do *cluster*. Simultaneamente, o RabbitMQ (versão 4.1.1) consolidou as *Quorum Queues* como padrão, utilizando o algoritmo de consenso Raft para garantir alta disponibilidade e integridade de dados. Entretanto, estudos que avaliem sistematicamente estas inovações ainda são escassos, sobretudo quando submetidas a diferentes escalas de volumetria de dados – desde dispositivos de borda, como câmeras, sensores, roteadores e servidores, localizados na periferia da rede, até *clusters* corporativos utilizados em grandes empresas.

Diante deste panorama, este trabalho propõe uma avaliação experimental das versões atualizadas do RabbitMQ e do Kafka. A comparação é conduzida contrastando ambas as

tecnologias com um cenário de linha de base (*baseline*) fundamentado em requisições HTTP (*Hypertext Transfer Protocol*), o protocolo fundamental que permite a comunicação entre clientes e servidores para transferir páginas e dados. O experimento avalia as implementações sob dois tamanhos de mensagens diferentes (*payload*) – 100 bytes e 1.000 bytes – e cinco perfis de carga progressivos – variando de 100 a 1.000.000 de mensagens – permitindo mensurar a vazão e a latência de cauda (percentis 95 e 99). A partir dos resultados obtidos, busca-se oferecer critérios objetivos para que arquitetos de *software* selecionem o *middleware* mais adequado aos requisitos de desempenho e escala de suas aplicações.

1.1 OBJETIVOS

A definição clara dos objetivos orienta o delineamento metodológico e garante que a comparação entre as tecnologias produza evidências válidas e reproduzíveis. Portanto, este estudo adota metas mensuráveis, detalhadas a seguir através do objetivo geral e dos objetivos específicos.

1.1.1 Objetivo geral

Realizar uma análise comparativa experimental de desempenho e escalabilidade entre os sistemas de mensageria RabbitMQ (versão 4.1.1 com *Quorum Queues*) e Apache Kafka (versão 4.0 com protocolo KRaft), confrontando-os com uma arquitetura síncrona de linha de base (*baseline*) para identificar o impacto da adoção de *middlewares* em aplicações submetidas a diferentes volumes de tráfego.

1.1.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Implementar um ambiente experimental padronizado e isolado via *containers* Docker, contendo três variantes arquiteturais da aplicação: uma síncrona (HTTP/REST), uma orientada a eventos com RabbitMQ e outra com Apache Kafka;
- Executar baterias de testes submetendo as três arquiteturas a cinco perfis de carga progressivos, variando de 100 (baixa escala) a 1.000.000 de mensagens (alta escala),

além de utilizar dois tamanhos de mensagens (100 bytes e 1.000 bytes) a fim de estressar os mecanismos de enfileiramento e persistência;

- Mensurar e contrastar as métricas de vazão e latência de cauda (percentis 95 e 99), evidenciando o comportamento de cada tecnologia tanto em cenários de ociosidade quanto de saturação;
- Sintetizar diretrizes objetivas que auxiliem na seleção da tecnologia de mensageria mais adequada, correlacionando os custos de latência observados com os requisitos de escala da aplicação.

1.2 JUSTIFICATIVA

O volume de eventos que trafegam em sistemas distribuídos tem aumentado exponencialmente, tornando os gargalos de comunicação uma causa recorrente de degradação de desempenho e indisponibilidade (SHARVARI; NAG, 2019). Embora os *message brokers* atuem na mitigação desses riscos através do desacoplamento entre emissores e receptores, a seleção do *middleware* adequado permanece um desafio arquitetural complexo. Diferentes soluções apresentam perfis distintos de vazão, latência e garantias de entrega, exigindo uma análise criteriosa de *trade-offs* (EUGSTER *et al.*, 2003).

Pesquisas anteriores, como a de Souza (2020), indicam que o Apache Kafka tende a alcançar maior vazão, enquanto o RabbitMQ oferece menor latência em cenários de roteamento complexo. No entanto, tais conclusões baseiam-se em versões anteriores a mudanças paradigmáticas recentes: a consolidação do protocolo de consenso KRaft no Apache Kafka, que elimina a dependência do ZooKeeper, e a padronização das *Quorum Queues* no RabbitMQ, que implementam o algoritmo Raft para tolerância a falhas. Na literatura atual, a avaliação sistemática dessas novas arquiteturas é escassa, deixando uma lacuna sobre a eficiência desses mecanismos modernos, principalmente, quando submetidos a diferentes escalas de carga.

Essa lacuna teórica reflete-se em desafios práticos na indústria. Aplicações sensíveis à latência, como plataformas de comércio eletrônico durante picos de acesso, frequentemente sofrem com a "latência de cauda", onde uma pequena porcentagem de requisições lentas prejudica a experiência do usuário final e impacta taxas de conversão (BARROSO; DEAN, 2013). Adicionalmente, estudos de confiabilidade, como o de Fu, Zhang e Yu (2021), demonstram que a perda de mensagens ainda ocorre quando configurações de replicação não são validadas sob estresse.

Portanto, justifica-se uma investigação que una as versões mais recentes das ferramentas, uma coleta de métricas rigorosa e cenários de escala variável. Para suprir essa necessidade, o presente trabalho oferece as seguintes contribuições:

- Avaliação de Arquiteturas Modernas: uma análise comparativa das versões mais recentes¹ do RabbitMQ (4.1.1 com *Quorum Queues*) e Apache Kafka (4.0 com KRaft), contrastando-as com um cenário de linha de base (*baseline*) síncrono para quantificar o ganho de escalabilidade proporcionado pelo modelo assíncrono.
- Evidências Empíricas em Múltiplas Escalas: uma prova de conceito que mensura vazão e latência de cauda em cinco perfis de carga progressivos (de 100 a 1.000.000 de mensagens), e dois diferentes tamanhos de mensagem (100 bytes e 1.000 bytes), permitindo identificar o ponto de inflexão onde cada tecnologia se torna mais vantajosa.
- Reprodutibilidade Experimental: a disponibilização de uma metodologia reprodutível, baseada em *containers* Docker e orquestração automatizada, acompanhada de roteiros de teste e código-fonte aberto em um repositório público do GitHub², contribuindo para que a comunidade acadêmica e industrial possa validar e estender os resultados apresentados.

Sendo assim, ao integrar resultados experimentais atuais a conceitos consolidados de sistemas distribuídos, este estudo pretende contribuir com dados empíricos atualizados para o corpo de conhecimento da área, fornecendo diretrizes objetivas para a adoção consciente de RabbitMQ³ ou Apache Kafka⁴ em projetos de diferentes escalas.

¹ Importa destacar que as versões especificadas (RabbitMQ 4.1.1 e Apache Kafka 4.0) constituem as versões mais recentes disponibilizadas à época da realização deste estudo (realizado entre Maio e Novembro de 2025). Ressalva-se que versões posteriores podem apresentar otimizações adicionais de desempenho.

² Disponível em: <<https://github.com/LeonardoFiamoncini/RMQ-and-Kafka-Comparative>>.

³ Documentação disponível em: <<https://www.rabbitmq.com/docs>>.

⁴ Documentação disponível em: <<https://kafka.apache.org/documentation/>>.

2 REVISÃO DE LITERATURA

A presente seção consolida conceitos fundamentais relacionados a sistemas distribuídos, filas de mensagens e replicação de dados. Estabelecer essa base teórica é necessário por dois motivos: primeiramente, assegura-se um vocabulário comum que evita ambiguidades na análise dos resultados experimentais; em segundo lugar, evidencia-se como as tecnologias analisadas respondem a limitações históricas apontadas na literatura. O percurso inicia-se pelos fundamentos gerais de distribuição, conceituando desafios inerentes à comunicação entre processos, para então aprofundar-se nas especificidades arquiteturais do RabbitMQ e do Apache Kafka.

2.1 SISTEMAS DISTRIBUÍDOS

Antes de analisar comparativamente as plataformas de mensageria, é imperativo retomar os princípios que regem aplicações executadas sobre múltiplos nós interligados. Nesta subseção, apresentam-se definições, motivações e desafios de sistemas distribuídos, fornecendo o contexto no qual *message brokers* se inserem como mecanismos de mitigação para gargalos de desempenho e tolerância a falhas.

2.1.1 Fundamentos e Aplicabilidade

Sistemas distribuídos consistem em uma coleção de computadores independentes que se apresentam aos seus usuários como um sistema único e coerente (TANENBAUM; VAN STEEN, 2017). Esta definição clássica engloba duas características basilares: a multiplicidade de componentes autônomos e a ilusão de unicidade, alcançada mediante mecanismos de transparência de acesso, localização e replicação.

A transparência é essencial, pois a coordenação entre processos distribuídos introduz complexidades inexistentes em sistemas centralizados, tais como latência de rede variável, concorrência e falhas parciais independentes (SILBERSCHATZ *et al.*, 2018). Do ponto de vista funcional, o objetivo primordial é permitir que aplicações cooperem para executar tarefas que excedem a capacidade individual de um nó, seja por exigirem paralelismo massivo, alta disponibilidade ou proximidade geográfica com os usuários.

A adoção de arquiteturas distribuídas tornou-se onipresente em domínios onde o volume, a velocidade ou a variabilidade dos dados superam os limites de processamento

vertical. Tanenbaum e Van Steen (2017) sintetizam três motivações principais para essa transição: compartilhamento de recursos, confiabilidade e escalabilidade. E para garantir, por exemplo, a confiabilidade em sistemas distribuídos sujeitos a falhas, é fundamental o conceito de consenso. É justamente com ele que permite-se que um conjunto de processos concorde sobre um valor (como a ordem de mensagens ou a liderança de um *cluster*), mesmo que alguns participantes falhem. Algoritmos como o Raft, utilizado tanto nas *Quorum Queues* do RabbitMQ quanto no protocolo KRaft do Kafka, foram projetados para tornar esse entendimento compreensível e implementável em sistemas práticos (ONGARO; OUSTERHOUT, 2014). Sendo assim, esses fatores justificam por que grandes corporações estruturam seus serviços sobre tais arquiteturas.

No setor de tecnologia, o Google popularizou o processamento em *clusters* massivos através do Google File System (GHEMAWAT *et al.*, 2003). No comércio eletrônico, a Amazon substituiu bancos de dados relacionais centralizados pelo Dynamo, um armazenamento chave-valor altamente disponível, focado em reduzir a latência em operações críticas de vendas (DECANDIA *et al.*, 2007). No domínio científico, o CERN modernizou sua infraestrutura de controle de aceleradores utilizando RabbitMQ para coordenar experimentos que geram grandes volumes de dados (ANDREASSEN *et al.*, 2017). Similarmente, microsserviços modernos, como descritos por Newman (2019), dependem de mensageria assíncrona para garantir que falhas em um componente não derrubem todo o sistema ("efeito dominó").

2.1.2 Comunicação em Sistemas Distribuídos

À luz desses cenários práticos e da necessidade do uso de sistemas distribuídos, fundamentada pela complexidade e volume de dados, levanta-se uma questão crucial: o modo como seus componentes se comunicam. De maneira geral, a comunicação entre processos em uma arquitetura distribuída pode ser classificada primariamente em síncrona ou assíncrona, sendo esta distinção crucial para a análise de desempenho em termos de latência e vazão. Esta classificação define o padrão de coordenação temporal entre os componentes (COULOURIS *et al.*, 2021).

2.1.2.1 Comunicação Síncrona

Neste modelo, o componente emissor (que, no caso, equivale ao cliente) envia uma requisição e bloqueia sua *thread* de execução, aguardando a resposta imediata do componente receptor (equivalente ao servidor) antes de prosseguir (TANENBAUM; VAN STEEN, 2017). A comunicação HTTP utilizada no *baseline* deste estudo, é o exemplo canônico de modelo síncrono. Sua principal limitação acaba recaindo sobre a baixa vazão em ambientes de alta concorrência, pois o tempo de processamento do cliente é diretamente dependente do RTT (*Round-Trip Time*) da rede, gerando ociosidade de recursos (NEWMAN, 2019). Sendo assim, a disponibilidade do sistema, em um modelo de comunicação síncrona, é dada pelo produto das disponibilidades de seus componentes. Ou seja, se o Serviço A depende sincronamente do Serviço B, a indisponibilidade de B torna A indisponível imediatamente (COULOURIS et al., 2021).

2.1.2.2 Comunicação Assíncrona

Neste modelo, o componente emissor envia a mensagem e continua imediatamente sua execução, sem esperar pela confirmação ou resposta do receptor. A garantia de entrega e o gerenciamento da fila de mensagens é delegada a um intermediário (no caso deste estudo, o *broker*) (COULOURIS et al., 2021). Neste tipo de comunicação, a principal vantagem acaba sendo o desacoplamento temporal (VIDELA; WILLIAMS, 2012). Nela, a arquitetura alcança alta vazão, pois o produtor pode injetar dados no sistema na velocidade máxima da rede sem ser restringido pela velocidade do consumidor (NEWMAN, 2019). Este padrão é a base das arquiteturas orientadas a eventos e mensagens.

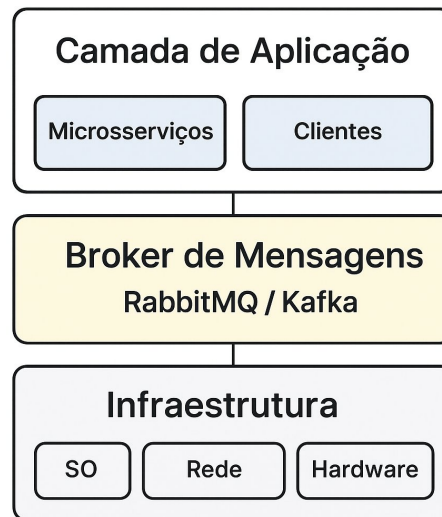
Dada a superioridade da vazão em modelos assíncronos para sistemas de grande porte, o foco deste trabalho recai sobre o *middleware* de mensagens como a tecnologia essencial para viabilizar este padrão de comunicação.

2.1.3 Arquitetura e Classificação por Escala

A arquitetura de um sistema distribuído estabelece como seus componentes de *software* interagem com a infraestrutura subjacente. De acordo com o modelo de referência de Coulouris *et al.* (2021), essa estrutura estratifica-se em três camadas fundamentais (Figura 1): a camada de aplicação, que encapsula a lógica de negócio; o *middleware*, responsável por prover abstrações de comunicação e persistência; e a infraestrutura, que engloba o *hardware* e a rede.

O *middleware* desempenha o papel central de ocultar a heterogeneidade da rede. É nesta camada que residem os *message brokers* (como RabbitMQ e Kafka), fornecendo a abstração de filas ou *logs* que permitem o desacoplamento temporal.

Figura 1 – Diagrama de camadas de um Sistema Distribuído



Fonte: Elaboração própria (2025)

Além da estratificação, Coulouris *et al.* (2021) classificam os sistemas em estilos arquiteturais, sendo o modelo Publicar/Assinar o mais relevante para este estudo. Nele, produtores emitem eventos sem conhecimento dos consumidores, delegando ao *middleware* a responsabilidade de roteamento e entrega (EUGSTER *et al.*, 2003).

Para fins de análise de desempenho, a classificação de um sistema distribuído baseia-se frequentemente na carga de trabalho que deve processar. Jain (1991) destaca que a taxa de chegada de requisições (medida em requisições por segundo ou mensagens por segundo) é o fator determinante para identificar gargalos. Sendo assim, baseando-se em Newman (2019), é possível categorizar aplicações em três perfis teóricos de carga:

1. Pequeno Porte: aplicações com baixa concorrência e tráfego esporádico (ex: painéis administrativos, automação residencial). Nestes casos, a latência de rede é raramente um problema crítico e soluções síncronas costumam ser suficientes.
2. Médio Porte: sistemas que processam fluxo constante de transações (ex: *e-commerce* regional). Newman (2019) observa que, neste nível, a comunicação síncrona começa a degradar a experiência do usuário, tornando essencial o uso de *brokers* para absorver picos de carga.

3. Grande Porte: aplicações de escala global (*Big Data*, *streaming* financeiro, redes sociais). Nestes cenários, descritos por Barroso e Dean (2013) como "escala de *warehouse*", o sistema deve lidar com milhões de eventos, onde a latência de cauda (P95/P99) e o particionamento de dados tornam-se os principais desafios de engenharia.

Embora essa taxonomia tripartite forneça um arcabouço conceitual sólido, a transição de desempenho entre esses estados não é discreta, mas contínua. Uma avaliação experimental que se limitasse a apenas um ponto de teste para cada categoria (um para pequeno, um para médio e um para grande) poderia falhar em identificar o momento exato em que a arquitetura satura.

Por essa razão, a metodologia deste trabalho refina essas três categorias teóricas em cinco cenários experimentais progressivos (denominados *Size 1* a *Size 5*). Essa granularidade adicional permite observar não apenas os estados estáveis, mas as zonas de fronteira entre eles: o *Size 1* estabelece a base para o pequeno porte; o *Size 2* investiga a transição para o médio porte; o *Size 3* consolida o comportamento típico de médio porte; o *Size 4* captura a transição para o grande porte; e o *Size 5* submete as tecnologias ao estresse máximo do grande porte. Essa abordagem visa identificar a curva de degradação do modelo síncrono e determinar o ponto de inflexão onde o *overhead* de um *broker* robusto como o Kafka se justifica frente à latência do RabbitMQ.

2.2 DESAFIOS EM SISTEMAS DISTRIBUÍDOS

A transição de uma arquitetura centralizada para um ambiente distribuído introduz complexidades que não podem ser negligenciadas. À medida que o volume de requisições cresce e a aplicação se fragmenta em múltiplos nós, surgem obstáculos que determinam diretamente a capacidade da solução de sustentar o *Service Level Agreement* (SLA). Esta seção examina os fatores críticos que degradam o desempenho e a confiabilidade, estabelecendo o pano de fundo teórico necessário para interpretar os resultados experimentais de vazão e latência apresentados posteriormente neste trabalho.

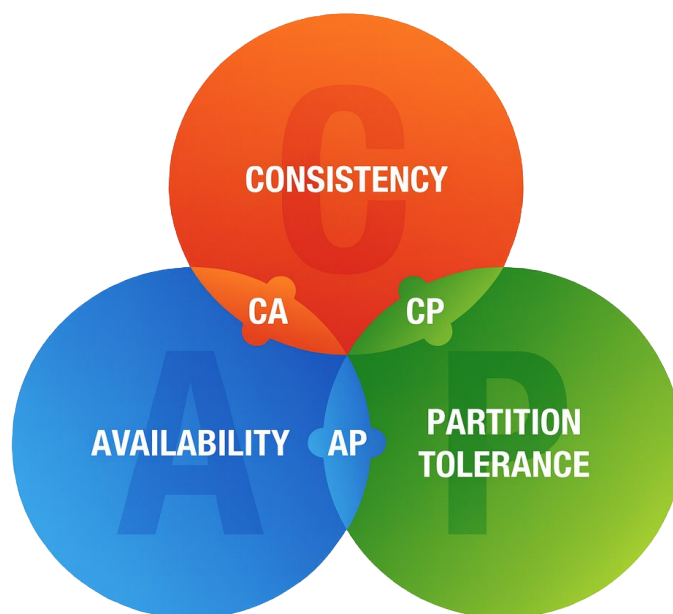
2.2.1 Falhas e Tolerância

A dificuldade fundamental em sistemas distribuídos reside na natureza independente das falhas de seus componentes. Tanenbaum e Van Steen (2017) definem esse fenômeno como "falha parcial": uma situação onde uma parte do sistema deixa de operar corretamente enquanto as demais continuam funcionando. Ao contrário de sistemas monolíticos, que tendem a falhar totalmente, sistemas distribuídos podem apresentar um estado de degradação parcial, o que dificulta o diagnóstico e o tratamento de erros. De maneira geral, as falhas manifestam-se predominantemente em três dimensões:

- Falhas de Processo: ocorrem quando um nó para de responder devido a *crashes* de *software*, esgotamento de recursos ou falhas de *hardware*.
- Falhas de Rede: incluem perda de pacotes, corrupção de dados ou latência excessiva, podendo resultar no isolamento de componentes (partição).
- Falhas de Temporização: acontecem quando uma resposta correta chega após o prazo limite (*timeout*) esperado pelo cliente, tornando-a inútil.

Nesse contexto, Gilbert e Lynch (2002) formalizaram o Teorema CAP (*Consistency, availability, partition tolerance*) (Figura 2), demonstrando que, em um sistema distribuído sujeito a partições de rede (P), é impossível garantir simultaneamente Consistência (C) e Disponibilidade (A). O arquiteto de *software* deve, assim, realizar uma escolha de *trade-off*.

Figura 2 – Teorema CAP



Fonte: Gerada por IA GPT 5 (2025)

Em arquiteturas estritamente síncronas, como a *baseline* adotada neste estudo, a instabilidade de um componente tende a propagar-se sistemicamente, resultando em falhas em cascata. Quando um serviço a jusante (*downstream*) apresenta degradação de desempenho, o serviço a montante (*upstream*) é forçado a manter conexões abertas e bloqueadas, exaurindo recursos críticos como *threads* e descritores de arquivo.

Barroso e Dean (2013) demonstram que, em sistemas de larga escala, mesmo um incremento marginal na latência de um componente individual amplifica exponencialmente a probabilidade de falha na operação global, fenômeno denominado "Latência de Cauda". Esse comportamento evidencia a necessidade arquitetural de mecanismos de desacoplamento, como filas de mensagens, que atuam isolando a falha do consumidor e preservando a disponibilidade do produtor.

2.2.2 Disponibilidade e Sobrecarga

A disponibilidade refere-se à proporção de tempo em que um sistema está operacional e capaz de processar requisições. Tanenbaum e Van Steen (2017) ressaltam que, estatisticamente, quanto maior o número de componentes em um sistema distribuído, menor a sua confiabilidade global, a menos que existam mecanismos explícitos de redundância.

A indisponibilidade não decorre apenas da paralisação total de um nó, mas frequentemente manifesta-se através da degradação severa de desempenho sob carga. Ou seja, o "ponto de saturação" do sistema: quando a taxa de chegada de requisições excede a capacidade de processamento, as filas internas de memória se esgotam, levando ao descarte de requisições e a erros de conexão.

Vogels (2009) argumenta que, para atingir a disponibilidade de cinco noves⁵, é imperativo adotar técnicas de consistência eventual e comunicação assíncrona, permitindo que o sistema continue aceitando escritas mesmo quando subsistemas de leitura ou processamento estiverem temporariamente inoperantes.

À vista disso, o papel dos *message brokers* transcende a simples entrega de dados; eles atuam como amortecedores de disponibilidade. Ao persistir mensagens em disco antes do processamento, tecnologias como Kafka e RabbitMQ permitem o "nivelamento de carga", assegurando que picos de tráfego não derrubem os consumidores e que janelas de manutenção possam ocorrer sem perda de dados.

⁵ Expressão comumente usada para descrever a confiabilidade de um sistema, indicando que o mesmo está operacional 99,999% do tempo.

2.3 RABBITMQ

Ao abordar as estratégias de mitigação para os problemas de comunicação distribuída, inicia-se pelo RabbitMQ, solução historicamente adotada quando os requisitos prioritários são o roteamento complexo e a garantia de confirmação individual de entrega. Esta seção estabelece o modelo de comunicação e as garantias de consistência da tecnologia, definições que servirão de base para a comparação, na sequência, com a abordagem de *log* distribuído utilizada pelo Apache Kafka.

2.3.1 Conceito e Fundamentos

O RabbitMQ é um *message broker* de código aberto que intermedia a troca assíncrona de mensagens, reduzindo o acoplamento direto característico das chamadas síncronas ponto a ponto. Desenvolvido na linguagem Erlang/OTP, plataforma reconhecida por sua capacidade de concorrência massiva e tolerância a falhas, o *broker* implementa nativamente o AMQP 0-9-1. Esta especificação define um padrão aberto para criação de filas, roteamento e confirmação de mensagens, garantindo interoperabilidade entre diferentes linguagens e sistemas operacionais (VIDELA; WILLIAMS, 2012).

Ao aderir estritamente ao protocolo AMQP, o RabbitMQ oferece semânticas de entrega robustas. O sistema suporta confirmações de recebimento (*acknowledgments*) que permitem garantias de "pelo menos uma vez" (*at-least-once*), onde a mensagem é re-enfileirada em caso de falha do consumidor, e "no máximo uma vez" (*at-most-once*), para dados efêmeros. Embora a garantia de "exatamente uma vez" (*exactly-once*) seja um desafio teórico em sistemas distribuídos, o RabbitMQ fornece as primitivas necessárias (como IDs de mensagem e confirmações manuais) para que os consumidores implementem processamento idempotente (BOSCHI; SANTOMAGGIO, 2013).

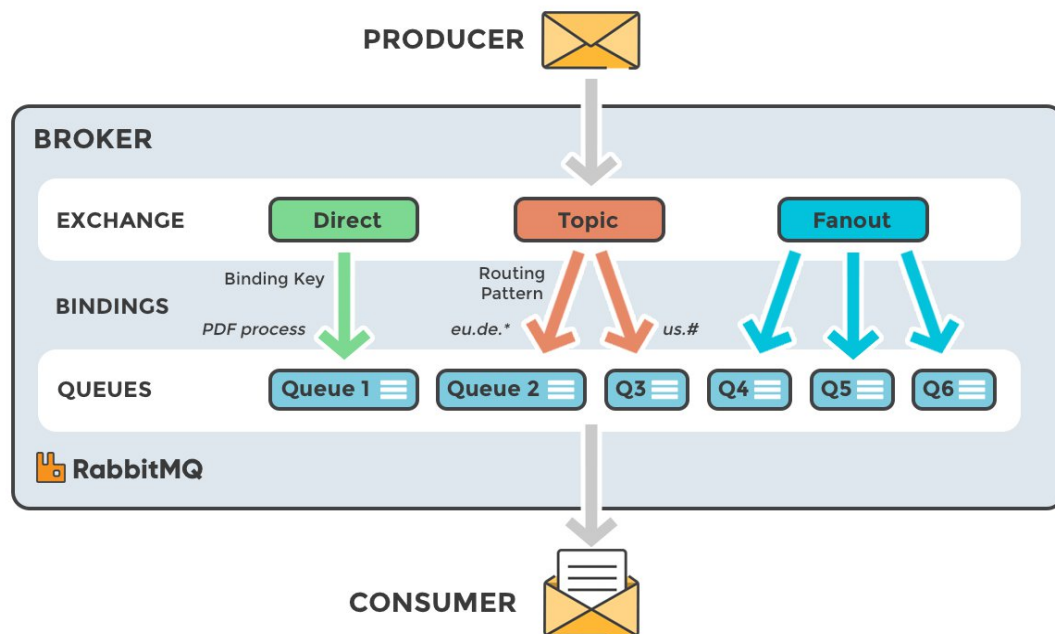
Desde sua concepção, o projeto evoluiu para suportar múltiplos protocolos através de uma arquitetura de *plugins*, incluindo MQTT (*Message Queuing Telemetry Transport*), por exemplo, ampliando sua aplicabilidade para cenários de *Internet das Coisas* e integração com sistemas legados. Essa flexibilidade consolidou sua adoção em setores como finanças e telecomunicações.

2.3.2 Arquitetura e Componentes

A arquitetura do RabbitMQ materializa o modelo de "Broker Inteligente, Consumidor Simples" (*Smart Broker, Dumb Consumer*). Em outras palavras, a lógica de roteamento reside centralizada no servidor, aliviando a complexidade das aplicações clientes. De acordo com Videla e Williams (2012), o fluxo de mensagens (Figura 3) opera através de três abstrações principais:

- *Exchanges* (Trocadores): componentes que recebem mensagens dos produtores e as roteiam baseados em regras predefinidas. Existem tipos distintos, como *Direct* (roteamento exato), *Fanout* (difusão para todas as filas) e *Topic* (padrões de correspondência).
- *Queues* (Filas): *buffers* de armazenamento onde as mensagens aguardam o consumo.
- *Bindings* (Ligações): regras que conectam *Exchanges* a *Queues*, definindo os critérios de roteamento.

Figura 3 – Fluxo de mensagens no RabbitMQ



Fonte: CloudAMQP (2025)

Em um ambiente de *cluster*, a consistência dos dados torna-se o desafio central. Historicamente, o RabbitMQ utilizava *Mirrored Queues* (filas espelhadas), cuja sincronização baseada em anel sofria de problemas de partição de rede, em que os nós divergiam sobre o estado da fila (MURATBEKOV, 2024). Para solucionar essa limitação, a versão 3.8

introduziu as *Quorum Queues*, que se tornaram o padrão a partir da versão 4.0. Essa estrutura implementa uma fila replicada baseada no algoritmo de consenso Raft (ONGARO; OUSTERHOUT, 2014).

Diferentemente do modelo anterior, as *Quorum Queues* garantem que uma mensagem só seja confirmada como "salva" após ser escrita em disco na maioria (quórum) dos nós do *cluster*. Esse mecanismo posiciona o RabbitMQ no lado da Consistência (CP) do Teorema CAP em casos de falha, priorizando a integridade dos dados em detrimento da disponibilidade parcial de escrita – característica fundamental analisada nos experimentos deste trabalho.

2.3.3 Aplicabilidade e Casos de Uso

A aplicabilidade do RabbitMQ é predominante em cenários que exigem roteamento sofisticado, priorização de mensagens e garantias transacionais de entrega. Embora frequentemente associado a sistemas de médio porte, sua arquitetura escala horizontalmente para lidar com volumes elevados, desde que o gargalo não seja a taxa de escrita sequencial em disco de uma única fila. Na literatura e na indústria, destacam-se casos de uso que exploram:

- **Infraestrutura Científica:** o CERN (*European Organization for Nuclear Research*) atualizou sua arquitetura de controle RADE utilizando RabbitMQ. Andreassen *et al.* (2017) documentam que a escolha foi motivada pela capacidade do *broker* de integrar protocolos distintos (C++ e Java via AMQP, dispositivos de *hardware* via MQTT) e pela resiliência das filas duráveis diante de reinicializações de serviço.
- **Serviços Financeiros:** instituições financeiras utilizam o mecanismo de *acknowledgment* manual para processamento de pagamentos. A garantia de que uma mensagem não é removida da fila até que o processamento seja confirmado evita perdas financeiras em caso de falha do consumidor, um requisito de conformidade regulatória.
- **Comércio Eletrônico:** plataformas de varejo utilizam o modelo de *Exchanges* para desacoplar o fechamento de um pedido de suas ações subsequentes. Um único evento de "Pedido Criado" pode ser roteado simultaneamente para filas de "Estoque", "Faturamento" e "Notificação", permitindo que cada subsistema escale de forma independente (VIDELA; WILLIAMS, 2012).

Em síntese, o RabbitMQ posiciona-se como um componente de resiliência que privilegia a consistência e a flexibilidade lógica. As filas Quorum, validadas na versão 4.1.1 utilizada neste estudo, reforçam essa posição ao oferecer tolerância a falhas baseada em consenso formal, mitigando os riscos históricos de perda de dados em sistemas distribuídos.

2.4 APACHE KAFKA

Concluída a análise do RabbitMQ, é necessário considerar um paradigma de mensageria projetado para maximizar a vazão sustentada e permitir a leitura de grandes volumes históricos – requisitos que emergem quando a aplicação ultrapassa os limites de processamento de um único nó e demanda persistência de longo prazo. Neste contexto, desponta o Apache Kafka, concebido não apenas como uma fila volátil, mas como uma plataforma distribuída de *streaming* de eventos, permitindo que múltiplos serviços consumam o mesmo dado em momentos distintos (reprocessamento) e que o histórico permaneça disponível para auditoria.

2.4.1 Conceito e Fundamentos

O Apache Kafka é um sistema distribuído de *logs* de *commit* particionados. Originalmente desenvolvido no LinkedIn para gerenciar o fluxo massivo de eventos e atividades da rede social, o projeto foi doado à Apache Software Foundation em 2011 (KREPS *et al.*, 2011). A proposta central do Kafka difere radicalmente dos *brokers* tradicionais. Em vez de focar na complexidade do roteamento (como as *exchanges* do AMQP), o Kafka foca na eficiência do armazenamento. De maneira simplista, ele estrutura os dados como *logs* imutáveis e anexáveis (*append-only logs*). Produtores gravam mensagens sequencialmente no final do *log*, enquanto consumidores leem esses registros em seu próprio ritmo, controlando seu próprio ponteiro (*offset*) de leitura (SHAPIRA *et al.*, 2017).

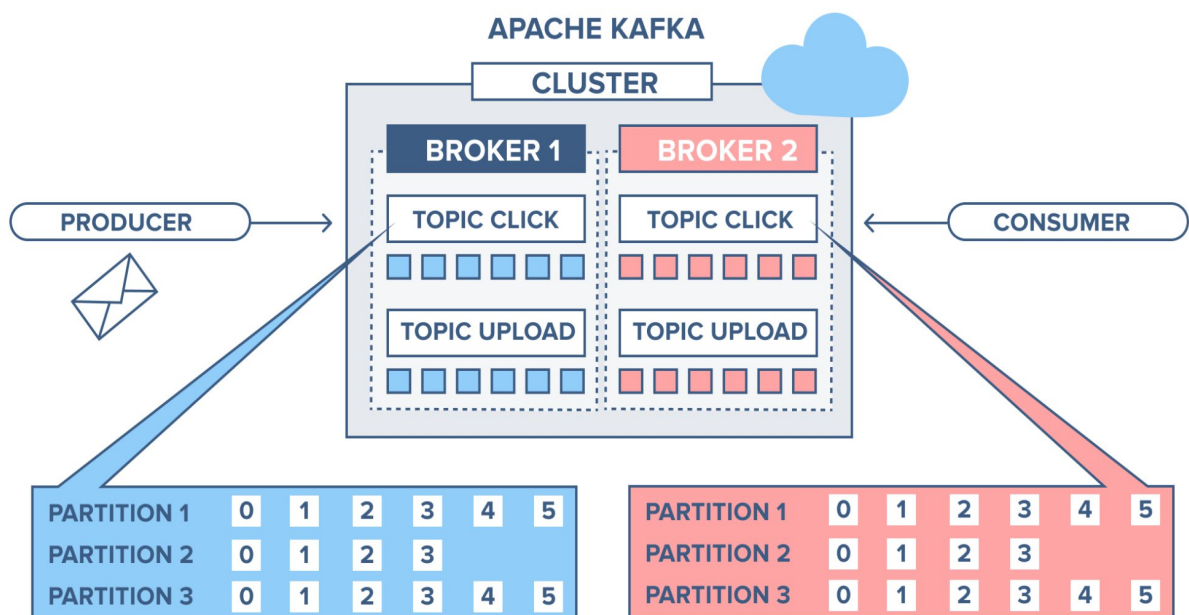
Diferentemente de sistemas de filas tradicionais que removem a mensagem imediatamente após o consumo, o Kafka persiste os dados em disco por um período configurável (retenção baseada em tempo ou tamanho). Este design viabiliza o desacoplamento temporal absoluto: um consumidor pode estar dias atrasado em relação ao produtor ou pode reiniciar a leitura do zero para reprocessar todo o histórico devido a uma alteração na regra de negócio (GONÇALVES, 2020). Portanto, ele atua como um barramento central nervoso para a integração de sistemas heterogêneos.

2.4.2 Arquitetura e Componentes

A arquitetura do Kafka é organizada em um *cluster* de servidores chamados *brokers*. A unidade lógica de dados é o Tópico, que é subdividido em Partições. A partição, por sua vez, é a unidade fundamental de paralelismo e escalabilidade. Segundo Kreps *et al.* (2011), cada partição é uma sequência ordenada de mensagens imutáveis. Ao dividir um tópico em N partições, o Kafka permite que o fluxo de gravação (Figura 4), tanto para escrita quanto para leitura, seja distribuído horizontalmente entre múltiplos servidores. A alta vazão do Kafka, observada nos experimentos de grande porte deste trabalho, deve-se a duas decisões de *design* de baixo nível descritas por Shapira *et al.* (2017):

- Entrada/Saída Sequencial: o Kafka evita leituras aleatórias em disco (*random seeks*), que são lentas. Ele restringe-se a gravações sequenciais e leituras lineares, aproveitando a pré-busca do sistema operacional.
- *Zero-Copy*: utiliza a chamada de sistema *sendfile* (em Linux) para transferir dados do *cache* de página diretamente para o *socket* de rede, evitando cópias desnecessárias de memória para o espaço do usuário.

Figura 4 – Fluxo de gravação no Apache Kafka



Fonte: CloudKafka (2020)

No que tange à consistência e coordenação, a versão 4.0 (utilizada neste estudo) marca uma mudança paradigmática com a consolidação do protocolo KRaft (*Kafka Raft Metadata*). Nas versões anteriores, o Kafka dependia de um sistema externo, o ZooKeeper, para gerenciar metadados do *cluster*, o que criava um gargalo de escalabilidade e complexidade operacional. Com o KRaft, o Kafka internaliza o gerenciamento de metadados utilizando um quórum de controladores baseados no algoritmo de consenso Raft (ONGARO; OUSTERHOUT, 2014). Isso simplifica a infraestrutura, melhora a estabilidade e reduz o tempo de recuperação em caso de falhas, permitindo a criação de milhões de partições com impacto reduzido no desempenho.

Para o consumo, o Kafka utiliza Grupos de Consumidores, onde, dentro de um grupo, cada partição é atribuída exclusivamente a um único consumidor. Isso garante a ordem de processamento dentro da partição e permite escalar a leitura linearmente: basta adicionar mais consumidores ao grupo (até o limite do número de partições) para aumentar a velocidade de processamento.

2.4.3 Aplicabilidade e Casos de Uso

A arquitetura do Kafka, apresentada anteriormente, brilha em cenários de alta volumetria (*Big Data*) e processamento de fluxo (*Stream Processing*), onde a latência de uma única mensagem é menos crítica do que a vazão total do sistema. Grandes corporações validam essa aplicabilidade em escala extrema:

- LinkedIn: berço da tecnologia, processa trilhões de mensagens diárias para alimentar recomendações e grafos de conexões sociais em tempo real (KREPS *et al.*, 2011).
- Uber: mantém uma das maiores implementações de Kafka do mundo para processar telemetria de corridas e transações financeiras. Yang (2022) relata que a arquitetura suporta picos de milhões de mensagens por segundo, essenciais para o cálculo de tarifas dinâmicas e estimativas de chegada.
- Indústria Automotiva: o BMW Group utiliza a plataforma para ingerir dados de sensores de milhões de veículos conectados. O Kafka atua como *buffer* de entrada, desacoplando a ingestão de dados em tempo real dos sistemas de *analytics* e aprendizado de máquina que operam em lote (BMW GROUP, 2021).

Dessa forma, o Kafka cobre o espectro de aplicações de "Grande Porte" definidas neste trabalho, oferecendo uma solução robusta onde a persistência, o reprocessamento e a vazão massiva são requisitos inegociáveis.

2.5 MÉTRICAS DE AVALIAÇÃO DE DESEMPENHO

A avaliação comparativa de sistemas distribuídos requer a adoção de métricas consolidadas que capturem não apenas a capacidade nominal de processamento, mas também a estabilidade do sistema sob estresse. Segundo Jain (1991), uma análise de desempenho robusta deve considerar múltiplas dimensões para evitar conclusões enviesadas por médias aritméticas que mascaram anomalias. Fundamentando-se nessa premissa, este estudo adota a latência de cauda (P95/P99) e a vazão como indicadores primários.

2.5.1 Latência e Percentis

A latência, no contexto deste trabalho, é definida como o tempo decorrido entre o instante em que uma mensagem é disponibilizada pelo produtor (T_{envio}) e o momento em que ela é totalmente recebida pelo consumidor ($T_{recebimento}$), configurando uma medição ponta a ponta.

Embora a média seja uma métrica comum, ela é insuficiente para sistemas distribuídos. Barroso e Dean (2013), em sua análise seminal "*The Tail at Scale*", demonstram que em arquiteturas de larga escala, a experiência do usuário é determinada pelos componentes mais lentos e enfatizam que a latência não deve ser analisada apenas através de valores médios. Em sistemas com milhares de transações, a experiência do usuário é frequentemente determinada pelos casos de pior desempenho. Dessa forma, para capturar esse comportamento, a literatura adota o uso dos percentis:

- Percentil 95 (P95): representa o valor de latência abaixo do qual se encontram 95% das observações. É uma métrica robusta para indicar o desempenho típico do sistema, filtrando *outliers* extremos e irrelevantes. Em outras palavras, essa métrica indica o desempenho típico do sistema sob condições normais, deixando de fora os 5% de requisições mais lentas.

- Percentil 99 (P99): abrange 99% das observações. Esta métrica é crítica para avaliar a estabilidade ("latência de cauda"), pois revela gargalos que afetam uma parcela pequena, porém significativa, das requisições. Segundo Barroso e Dean (2013), em sistemas que dependem de múltiplas chamadas paralelas, a probabilidade de pelo menos uma requisição lenta afetar o resultado final aumenta exponencialmente, tornando o controle do P99 essencial para atingir objetivos de nível de serviço (SLOs).

2.5.2 Vazão

A métrica de vazão⁶ quantifica a capacidade de um sistema processar mensagens por unidade de tempo, sendo tradicionalmente mensurada em mensagens por segundo (msg/s). Jain (1991) define vazão como a taxa máxima na qual um sistema pode processar trabalho sem exceder limites de desempenho predefinidos.

A relação entre vazão e latência é fundamental na análise de desempenho. Isso se deve pois, conforme a vazão se aproxima da capacidade máxima de saturação do sistema, as filas internas crescem, elevando a latência de forma não linear, devido a efeitos de contenção e coerência. Este fenômeno é descrito pela Lei de Utilização, que, em outras palavras, estabelece que a latência aumenta assintoticamente conforme a utilização do sistema se aproxima de 100%. Sendo assim, a avaliação de um *message broker* exige a observação conjunta dessas duas métricas: um sistema ideal mantém alta vazão sem degradar severamente a latência de cauda.

⁶ O termo "vazão" equivale ao inglês *throughput*, amplamente difundido na literatura técnico-científica. Ambos os termos são tratados como sinônimos e empregados de forma intercambiável por diversos autores da área.

3 METODOLOGIA

A presente pesquisa adota uma abordagem metodológica experimental caracterizada pela implementação prática de cenários de *benchmark* controlados. O delineamento do método segue os princípios de avaliação de desempenho de sistemas computacionais estabelecidos por Jain (1991), com ênfase estrita na reprodutibilidade dos cenários, no isolamento de variáveis de confusão e na análise estatística de métricas de QoS (*Quality of Service*).

A estratégia experimental consiste na instrumentação e análise comparativa de três paradigmas arquiteturais de comunicação: uma síncrona (*baseline*) e duas implementações assíncronas baseadas nos *middlewares* RabbitMQ e Apache Kafka. Tais arquiteturas foram submetidas a cargas de trabalho progressivas a fim de isolar e quantificar o impacto da escolha tecnológica nas métricas de latência e vazão.

3.1 AMBIENTE EXPERIMENTAL

Para assegurar a validade interna dos experimentos e a consistência dos dados coletados, optou-se pela condução de um experimento controlado em ambiente isolado, em detrimento de uma implantação em nuvem pública (ambiente real), e padronizado utilizando virtualização em nível de sistema operacional (containerização). Essa escolha visa maximizar a validade interna do estudo, eliminando variáveis de confusão (ruído) como latência variável de *internet*, vizinhos barulhentos em ambientes virtualizados e oscilações de provedores, conforme recomendado por Jain (1991) para a caracterização *baseline* de desempenho de *software*. Além disso, todos os componentes da arquitetura (produtores, consumidores e *brokers*) foram orquestrados em uma única máquina física, comunicando-se através de uma rede *bridge* virtual dedicada.

Um controle experimental adotado neste estudo foi o isolamento total da interface de rede externa. Durante a execução das baterias de testes, a estação de trabalho operou desconectada de quaisquer redes externas (*internet* e rede local). Este procedimento teve como objetivo eliminar a variabilidade introduzida por tráfego de fundo não determinístico – como atualizações automáticas de sistema, telemetria ou oscilações de sinal em conexões sem fio. Dessa forma, garante-se que os gargalos e latências observados sejam decorrentes exclusivamente das limitações intrínsecas do *software* (*overhead* dos protocolos/*brokers*) e da

capacidade de processamento do *hardware* local, isentando os resultados de interferências exógenas de roteamento.

3.1.1 Especificações de *Hardware* e *Software*

Os experimentos foram conduzidos em uma estação de trabalho cujo *hardware* conta com as seguintes características:

- Processador: Intel® Core™ 7 240H (16 *threads*, até 5.0 GHz);
- Memória RAM (*Random Access Memory*): 32 GB DDR5;
- Armazenamento: SSD (*Solid State Drive*) NVMe 512 GB;
- GPU dedicada: NVIDIA GeForce RTX 3050 6 GB;
- Sistema Operacional: Ubuntu 24.04.3 LTS (64 bits);
- Kernel: Linux 6.14.0-1016-oem;
- Virtualização: Docker Engine 24.0+ com Docker Compose 2.20+.

3.1.2 Definição e Configuração dos *Brokers*

As versões dos *softwares* selecionadas para a realização dos testes são as mais atuais até a data deste trabalho em questão, todavia, foram fixadas para garantir a reprodutibilidade histórica. Tendo isso em mente, segue abaixo a descrição de cada uma das três variantes da aplicação implementadas:

1. *Baseline* (HTTP/REST): a implementação síncrona utiliza o *framework* Flask (versão 3.1.1), operando sobre um servidor WSGI (*Web Server Gateway Interface*) *multi-thread* para suportar concorrência de rede.
2. RabbitMQ (Versão 4.1.1): foi configurado utilizando exclusivamente *Quorum Queues*, estrutura baseada no algoritmo de consenso Raft para garantir alta disponibilidade e integridade dos dados.
3. Apache Kafka (Versão 4.0.0): foi operado em modo KRaft (Kafka Raft), eliminando a dependência externa do ZooKeeper e centralizando a gestão de metadados no próprio *cluster* de *brokers*.

3.2 DELINEAMENTO DOS CENÁRIOS DE TESTE

Para avaliar a escalabilidade das soluções, definiu-se uma matriz de testes composta por cinco perfis de carga progressivos, denominados “*Sizes*”. Diferentemente de abordagens que fixam a taxa de requisições por segundo (RPS), este estudo optou por fixar o volume total de mensagens a serem processadas em lote, medindo o tempo necessário para a conclusão da tarefa. Essa abordagem, recomendada por Jain (1991), permite observar o comportamento de saturação de forma mais clara. Com base nesse critério, os cenários foram definidos como:

- *Size 1* (Carga Mínima): envio de 100 mensagens.
- *Size 2* (Carga Leve): envio de 1.000 mensagens.
- *Size 3* (Carga Média): envio de 10.000 mensagens.
- *Size 4* (Carga Alta): envio de 100.000 mensagens.
- *Size 5* (Carga Massiva): envio de 1.000.000 de mensagens.

Além da variação no volume de requisições, este estudo introduz a variável de tamanho da mensagem para avaliar o comportamento do sistema sob diferentes densidades de dados e custos de serialização. A definição dos tamanhos de carga útil (*payload*) fundamenta-se em cenários de uso típicos identificados na literatura:

- 100 bytes (Perfil de Eventos Leves): tamanho selecionado com base no desenho experimental de Kreps *et al.* (2011). Em seus testes seminais, os autores utilizaram uma faixa reduzida (200 bytes) para simular registros de *log* e eventos de telemetria, nos quais o custo de processamento do cabeçalho do protocolo tende a ser mais significativo do que a transmissão dos dados em si. Com base nessa referência, adotou-se 100 bytes para estressar o cenário de pequenos eventos neste trabalho.
- 1.000 bytes (Perfil Transacional Padrão): adotado como o padrão de referência no estudo comparativo de Nikhil e Chandar (2020), o tamanho de 1 KB representa a carga típica de microsserviços trafegando objetos de negócio estruturados JSON (*JavaScript Object Notation*), sendo considerado o "padrão ouro" para testes de capacidade de canal.

Dessa forma, o experimento totaliza 30 cenários de teste distintos (3 combinações de tecnologias $\times$ 5 tamanhos de carga $\times$ 2 tamanhos de mensagem), sendo cada um dos 30 cenários executados uma vez iterativamente de forma consecutiva.

3.3 INSTRUMENTAÇÃO E COLETA DE DADOS

A validade dos resultados em uma análise de desempenho depende diretamente da precisão dos instrumentos de medição e da minimização da interferência causada pela própria coleta de dados, fenômeno conhecido como "efeito de sonda" (*probe effect*). Para mitigar tais interferências e garantir a fidedignidade dos registros, desenvolveu-se um módulo de telemetria dedicado, integrado diretamente ao código-fonte da aplicação, operando sob três pilares técnicos: sincronização temporal, instrumentação de carga útil e cálculo estatístico.

3.3.1 Sincronização e Precisão Temporal

A sincronização precisa de relógios físicos em sistemas distribuídos é, notoriamente, um dos desafios mais complexos da computação distribuída, visto que relógios de *hardware* tendem a divergir (*drift*) ao longo do tempo (TANENBAUM; VAN STEEN, 2017). Contudo, dado que o ambiente experimental foi projetado sobre uma arquitetura de *containers* Docker hospedados em um único *host* físico, tirou-se proveito do compartilhamento do *kernel* do sistema operacional. Isso permitiu a utilização do relógio monotônico de alta resolução (*time.perf_counter* em Python) para cálculos de duração, garantindo precisão na escala de nanossegundos e imunidade a ajustes de horário do sistema (*clock skew*), conforme recomendado por Jain (1991) para *benchmarks* de alta precisão.

Para a marcação temporal absoluta (*timestamping*), utilizou-se o tempo de sistema (*time.time*), encapsulado diretamente na estrutura da mensagem. O protocolo de medição define a latência (L) de cada mensagem i como a diferença entre o instante de recebimento pelo consumidor (t_{Rx}) e o instante de envio pelo produtor (t_{Tx}), formalizado pela equação:

$$L_i = t_{Rx,i} - t_{Tx,i}.$$

É importante ressaltar que t_{Tx} é capturado imediatamente antes da chamada de rede no produtor, e t_{Rx} imediatamente após a disponibilidade da mensagem na memória do

consumidor, caracterizando uma medição ponta a ponta que engloba o tempo de serialização, transmissão, enfileiramento no *broker* e desserialização.

3.3.2 Estrutura de Dados e Persistência

Cada mensagem trafegada no sistema carrega um *payload* padronizado em formato JSON, para garantir legibilidade e facilidade de *parsing*, contendo um identificador único universalmente exclusivo UUID (*Universally Unique Identifier*) e o *timestamp* de origem. Para as tecnologias RabbitMQ e *Baseline*, o formato da mensagem utilizada foi:

```
{
  "id": "msg_1",
  "timestamp": 1732881705.123456,
  "body": "xxx..."
}
```

Já para o Apache Kafka, foi utilizado um atributo a mais nomeado “*run_id*”. Com ele, garante-se que cada execução processe apenas suas próprias mensagens, evitando contaminação por mensagens antigas e garantindo métricas precisas e isoladas por teste. Dito isso, a estrutura das mensagens sendo enviadas para o Apache Kafka foi:

```
{
  "id": "msg_1",
  "run_id": "kafka-size3-1732881705-442053", # Identificador único
  "timestamp": 1732881705.123456,
  "body": "xxx..."
}
```

Além disso, a classe coletora de métricas armazena os registros de latência em memória durante a execução do lote para evitar a latência de Entrada/Saída de disco a cada mensagem. Somente após a conclusão do processamento de todo o lote, os dados brutos são consolidados e persistidos em arquivos no formato CSV (*Comma-Separated Values*).

Essa abordagem de persistência em lote assegura que o processo de escrita de *logs* não compete por recursos de disco com o *message broker* durante o teste crítico, preservando

o isolamento do experimento. Os arquivos gerados contêm a série temporal completa das latências, permitindo auditorias posteriores e a reconstrução exata dos gráficos de distribuição.

3.3.3 Tratamento Estatístico e Percentis

Para a análise dos dados coletados, rejeitou-se o uso exclusivo da média aritmética, visto que esta métrica é sensível a valores extremos e incapaz de representar a experiência do usuário em sistemas de cauda longa. Sendo assim, a análise estatística fundamentou-se no cálculo dos quantis amostrais, especificamente os percentis 95 (P95) e 99 (P99).

Esse cálculo dos percentis foi implementado utilizando o Método 7 de Hyndman e Fan (1996), que é o padrão adotado pelas principais bibliotecas de computação científica (como *NumPy*). Este método utiliza a interpolação linear entre observações ordenadas para estimar o valor do quantil quando a posição calculada não corresponde a um índice inteiro. A adoção desse método formal garante que os resultados sejam matematicamente rigorosos e comparáveis com outros estudos da área.

3.4 PROCEDIMENTO DE EXECUÇÃO

Para assegurar a reprodutibilidade e a consistência interna dos experimentos, definiu-se um protocolo de execução orquestrado por *scripts* de automação, representado pelo arquivo *execute_all.sh*, que está disponível no repositório público do GitHub⁷. O procedimento para cada um dos cenários de carga segue uma sequência de passos projetada para eliminar estados residuais de execuções anteriores.

Inicialmente, realiza-se a etapa de "limpeza e aquecimento". Nela, antes de cada bateria de testes, os *containers* dos *brokers* são reiniciados para limpar *caches* de memória e descritores de arquivos. No caso específico do Apache Kafka, o tópico de teste é excluído e recriado programaticamente para garantir que não haja segmentos de *log* antigos ocupando espaço em disco ou influenciando o desempenho de Entrada/Saída. Já para o RabbitMQ, as filas são purgadas e reinicializadas.

Após a estabilização dos serviços, cuja verificação foi realizada através de *health checks*⁸, inicia-se a execução da carga de trabalho. O orquestrador dispara o processo

⁷ Disponível em: <<https://github.com/LeonardoFiamoncini/RMQ-and-Kafka-Comparative>>.

⁸ *Health Checks* (Verificações de Integridade) são mecanismos automatizados que consultam periodicamente o

consumidor, que aguarda em estado de prontidão, seguido pelo disparo do produtor que injeta a carga definida. Feito isso, o sistema aguarda a confirmação de recebimento de todas as mensagens ou o encerramento por *timeout* de segurança, onde foi definido um valor padrão (configurável) de 30s. Nesse contexto, vale ressaltar que, durante a execução dos experimentos, não foram registradas ocorrências de *timeout*.

Ao final de cada execução, as métricas de vazão – calculadas como a razão entre o total de mensagens e o tempo delta de execução – e os vetores de latência são exportados. Esse ciclo é repetido sequencialmente para todas as tecnologias (*Baseline*, RabbitMQ e Kafka) e todos os tamanhos de carga, sem intervenção manual, garantindo que todas as variantes sejam submetidas às mesmas condições ambientais da máquina hospedeira.

4 TRABALHOS CORRELATOS

A análise da literatura revela um corpo substancial de trabalhos dedicados à comparação de sistemas de mensageria, refletindo a importância crítica desses componentes na engenharia de *software* moderna. No entanto, observa-se que a rápida evolução dessas tecnologias frequentemente torna obsoletos os resultados empíricos prévios. Sendo assim, apesar da literatura contar com diversas contribuições relevantes, inúmeras lacunas tecnológicas e metodológicas, que este trabalho busca preencher, são identificadas.

O trabalho de Souza (2020) estabeleceu bases importantes para a comparação direta entre Apache Kafka e RabbitMQ. O autor demonstrou, através de cenários de carga sintética, a superioridade do Kafka em vazão para grandes volumes de dados, enquanto o RabbitMQ apresentou latências inferiores em cenários de fila única com baixa concorrência. Embora conclusivo para o contexto da época, o estudo limitou-se a versões que dependiam fortemente do ZooKeeper (no caso do Kafka) e de filas espelhadas clássicas (no RabbitMQ), arquiteturas que diferem substancialmente dos mecanismos de consenso modernos.

Na mesma linha, Raje (2019) investigou o impacto das configurações de persistência, demonstrando que escolhas arquiteturais – como a escrita síncrona em disco versus o uso de *cache* em memória – podem inverter as vantagens comparativas entre os *brokers*. Sharvari e Nag (2019) expandiram essa análise para incluir o NATS *Streaming*, reforçando que o Kafka tende a ser a escolha dominante para pipelines de dados onde a retenção de histórico é prioritária, em detrimento da latência em tempo real.

estado de um serviço para determinar se ele está apto a receber requisições. No contexto deste trabalho, *scripts* aguardavam a resposta '200 OK' ou 'Ready' dos *containers* antes de iniciar os testes.

Além disso, Dobbelaere e Esmaili (2017) conduziram um estudo comparativo seminal avaliando Kafka e RabbitMQ. Nele, os autores concluíram que o Kafka oferece maior vazão em detrimento de latência em cenários de *batching*, enquanto o RabbitMQ apresenta melhor latência em cargas leves. Entretanto, o estudo foi realizado sobre versões antigas (Kafka 0.10 e RabbitMQ 3.6), anteriores à introdução de algoritmos de consenso robustos como o Raft, o que limita a aplicabilidade direta de seus resultados para arquiteturas modernas de alta disponibilidade.

Mais recentemente, Arruda e Martins (2020) apresentaram uma análise de desempenho focada no Instituto Federal do Sudeste de Minas Gerais. Embora com uma metodologia similar, ele focou em cenários de integridade padrão, o que acaba diferenciando o presente trabalho ao isolar especificamente o impacto das configurações de Alta Disponibilidade em escalas como de 1.000.000 de mensagens, por exemplo, e ao incluir um *baseline* HTTP para justificar o custo arquitetural da adoção de *brokers*.

Outra contribuição seminal para o rigor metodológico foi apresentada por Fu, Zhang e Yu (2021). Os autores criticaram *benchmarks* enviesados que comparavam configurações desiguais (ex: Kafka com persistência assíncrona versus RabbitMQ com persistência forte). Eles propuseram um *framework* padronizado ("*Fair Comparison*"), enfatizando a necessidade de equalizar garantias de durabilidade e tamanho de *payload* para obter resultados válidos. O presente trabalho adota premissas similares, garantindo que tanto o RabbitMQ (via *Quorum Queues*) quanto o Kafka (via replicação) operem com níveis comparáveis de segurança de dados.

No que tange aplicações em domínios específicos, a literatura também explora o comportamento dessas ferramentas em cenários de nicho. Milošavljević *et al.* (2021) avaliaram *brokers* em ambientes de *Internet das Coisas* e *Smart Homes*, constatando que o Kafka escala horizontalmente de forma mais eficiente quando há milhares de tópicos ativos. Porém, Dunne *et al.* (2018) focaram em sistemas embarcados, concluindo que o RabbitMQ oferece maior previsibilidade temporal, característica crítica para sistemas de tempo real.

Contudo, sua análise permaneceu majoritariamente qualitativa, carecendo da validação estatística de cauda longa (percentil 99) proposta por Barroso e Dean (2013). Com isso, a revisão crítica dos trabalhos supracitados evidencia três lacunas principais que justificam a realização desta nova pesquisa experimental:

- Obsolescência dos Mecanismos de Consenso: nenhum dos estudos revisados avalia as versões atuais dos *brokers* operando sob seus novos paradigmas de consenso

distribuído. Especificamente, faltam dados sobre o desempenho do Apache Kafka 4.0 rodando sob o protocolo KRaft (sem ZooKeeper) comparado ao RabbitMQ 4.1.1 utilizando *Quorum Queues* (Raft). Essas mudanças alteram fundamentalmente o comportamento de Entrada/Saída e latência.

- Ausência de *Baseline* Síncrono: a maioria dos comparativos foca exclusivamente na disputa "*Broker A vs. Broker B*", negligenciando a comparação com uma arquitetura sem *broker* (HTTP/REST). A inclusão de um *baseline* é essencial para quantificar o *overhead* introduzido pelo *middleware* e justificar o custo arquitetural de sua adoção em aplicações de pequeno porte.
- Análise de Cauda (*Tail Latency*): muitos estudos anteriores concentram-se em médias aritméticas de latência. Este trabalho avança ao focar nos percentis 95 e 99, métricas que refletem com maior fidelidade a estabilidade do sistema e a experiência do usuário em cenários de estresse, conforme as recomendações modernas de engenharia de confiabilidade.

Com isso, em síntese, o diferencial deste trabalho frente aos estudos supracitados e *benchmarks* informais reside no rigor metodológico aplicado às versões das tecnologias (mais recentes até o momento do presente estudo, em novembro de 2025), que eliminam dependências externas como o ZooKeeper, alterando fundamentalmente o comportamento de Entrada/Saída dos sistemas.

5 DESENVOLVIMENTO

Este capítulo apresenta a consolidação e a análise crítica dos dados obtidos através da execução experimental dos cenários de *benchmark* definidos na metodologia. O objetivo central é confrontar as hipóteses teóricas sobre o comportamento dos sistemas de mensageria – RabbitMQ e Apache Kafka – e do modelo síncrono diante de diferentes escalas de carga.

A avaliação foi conduzida em um ambiente submetendo as três variantes arquiteturais a cinco perfis de carga, denominados neste contexto de *Sizes*, que progridem de 100 mensagens a 1.000.000 de mensagens, além de variar no tamanho da mensagem sendo enviada (100 bytes e 1.000 bytes). Fora isso, a implementação técnica, conforme detalhado no capítulo anterior, utilizou instrumentação de código para capturar métricas ponta a ponta. Isso assegura que os tempos registrados englobam todo o ciclo de vida da mensagem: desde a serialização no produtor, passando pela persistência no *broker* (com confirmação de quórum ou replicação), até a desserialização final no consumidor.

5.1 CONSOLIDAÇÃO DOS DADOS OBTIDOS

A visão geral do desempenho das tecnologias avaliadas pode ser observada nas Tabelas 1 e 2, que sintetizam as métricas de vazão e latência de cauda (P95 e P99) para todos os tamanhos de carga testados, segmentando de acordo com o tamanho da mensagem – 100 e 1.000 bytes, respectivamente. Para uma análise precisa, todos os valores exatos coletados durante os experimentos estão detalhados nas tabelas. A partir delas, observa-se uma clara distinção no comportamento das tecnologias à medida que o volume de dados aumenta, evidenciando os *trade-offs* de design de cada solução.

Tabela 1 – Resultados Experimentais para Mensagens de 100 bytes

Cenários	Tecnologia	Vazão (msg/s)	Latência P95 (ms)	Latência P99 (ms)
Size 1 (100 msg/s)	Baseline	425.5	2.4	2.6
	RabbitMQ	96.2	12.9	13.0
	Apache Kafka	15.9	2142.8	2144.6
Size 2 (1.000 msg/s)	Baseline	435.5	2.4	2.5
	RabbitMQ	793.0	20.4	21.8
	Apache Kafka	162.2	2028.2	2035.5

Size 3 (10.000 msgs)	Baseline	436.9	2.4	2.5
	RabbitMQ	2699.9	196.5	212.4
	Apache Kafka	1562.7	2112.1	2201.9
Size 4 (100.000 msgs)	Baseline	436.4	2.4	2.5
	RabbitMQ	4065.7	4803.8	4853.4
	Apache Kafka	4678.1	1924.5	2353.1
Size 5 (1.000.000 msgs)	Baseline	435.1	2.4	2.5
	RabbitMQ	3603.8	8295.9	9869.5
	Apache Kafka	5285.3	1928.2	2051.4

Fonte: Elaboração própria (2025)

Tabela 2 – Resultados Experimentais para Mensagens de 1.000 bytes

Cenários	Tecnologia	Vazão (msg/s)	Latência P95 (ms)	Latência P99 (ms)
Size 1 (100 msgs)	Baseline	427.5	2.5	2.8
	RabbitMQ	96.3	15.7	15.7
	Apache Kafka	16.3	2022.4	2024.4
Size 2 (1.000 msgs)	Baseline	436.1	2.4	2.5
	RabbitMQ	788.5	19.6	22.0
	Apache Kafka	160.2	2067.2	2099.9
Size 3 (10.000 msgs)	Baseline	437.3	2.4	2.5
	RabbitMQ	2114.0	1175.6	1176.5
	Apache Kafka	1287.2	2482.0	2727.6
Size 4 (100.000 msgs)	Baseline	435.0	2.4	2.5
	RabbitMQ	2822.9	6364.4	6455.5
	Apache Kafka	3079.0	1563.1	2435.3
Size 5 (1.000.000 msgs)	Baseline	435.1	2.4	2.5
	RabbitMQ	2734.4	8642.0	9912.9
	Apache Kafka	3603.9	2053.8	2091.7

Fonte: Elaboração própria (2025)

Uma leitura preliminar desses dados revela três tendências macroscópicas fundamentais que nortearão a discussão nas próximas seções:

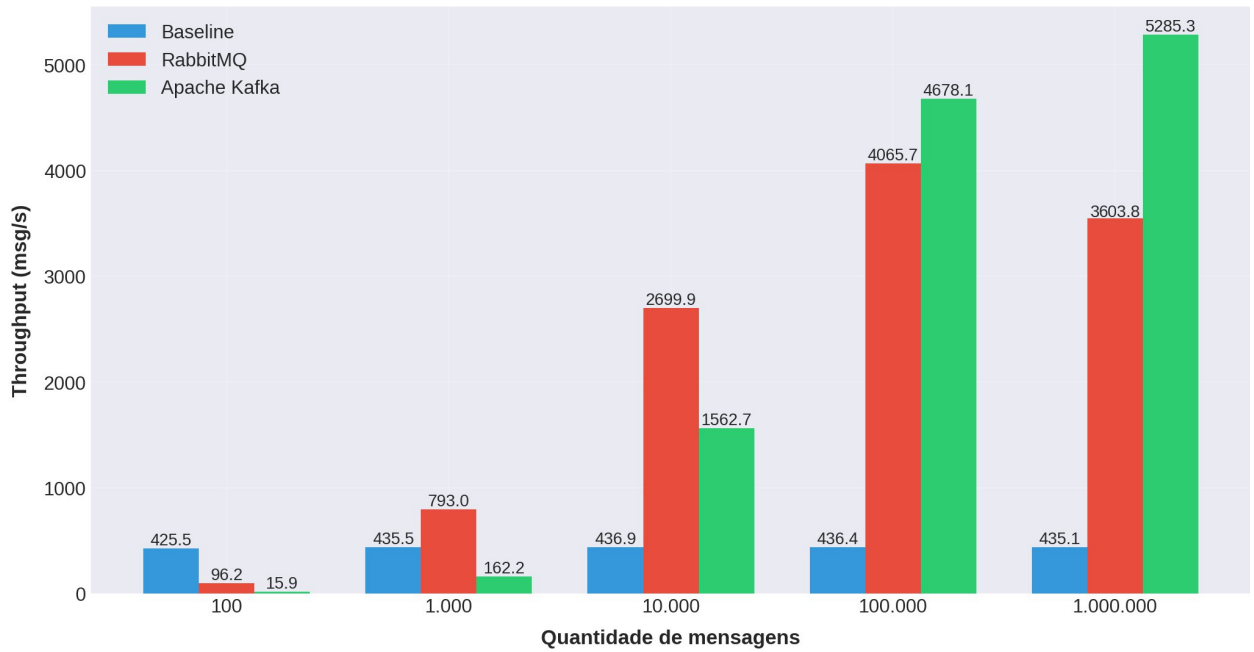
1. O Teto de Processamento do Modelo Síncrono: o *Baseline* HTTP demonstra uma consistência notável em latência (sempre abaixo de 3ms), porém sofre de um teto severo de vazão, estagnando em aproximadamente 430 msg/s independentemente do tamanho da carga. Com isso, prova-se que o gargalo é o *overhead* de conexão e não o tamanho do *payload* em si.
2. A Eficiência do RabbitMQ em Cargas Intermediárias: o RabbitMQ apresentou o melhor equilíbrio geral para cargas intermediárias, principalmente no cenário de 10.000 mensagens (*Size* 3), combinando alta vazão com latência considerável. Apesar disso, ele sofreu uma queda de desempenho perceptível com o aumento da mensagem a partir do *Size* 4, momento este marcado justamente pela virada do Apache Kafka.
3. O Custo de Inicialização e a Virada do Kafka: o Apache Kafka exibe um desempenho contra-intuitivo em cargas baixas (*Sizes* 1 e 2), com latências médias superiores a 2 segundos e baixa vazão. No entanto, observa-se um ponto de inflexão a partir do *Size* 4, onde ele supera o RabbitMQ em vazão, atingindo seu pico de eficiência no cenário de 1.000.000 de mensagens, o que valida a hipótese da sua arquitetura otimizada para fluxo de dados.

As seções subsequentes dissecam esses comportamentos, correlacionando-os com as características arquiteturais de bloqueio de Entrada/Saída, algoritmos de consenso e estratégias de bufferização discutidas no referencial teórico.

5.2 ANÁLISE DE VAZÃO

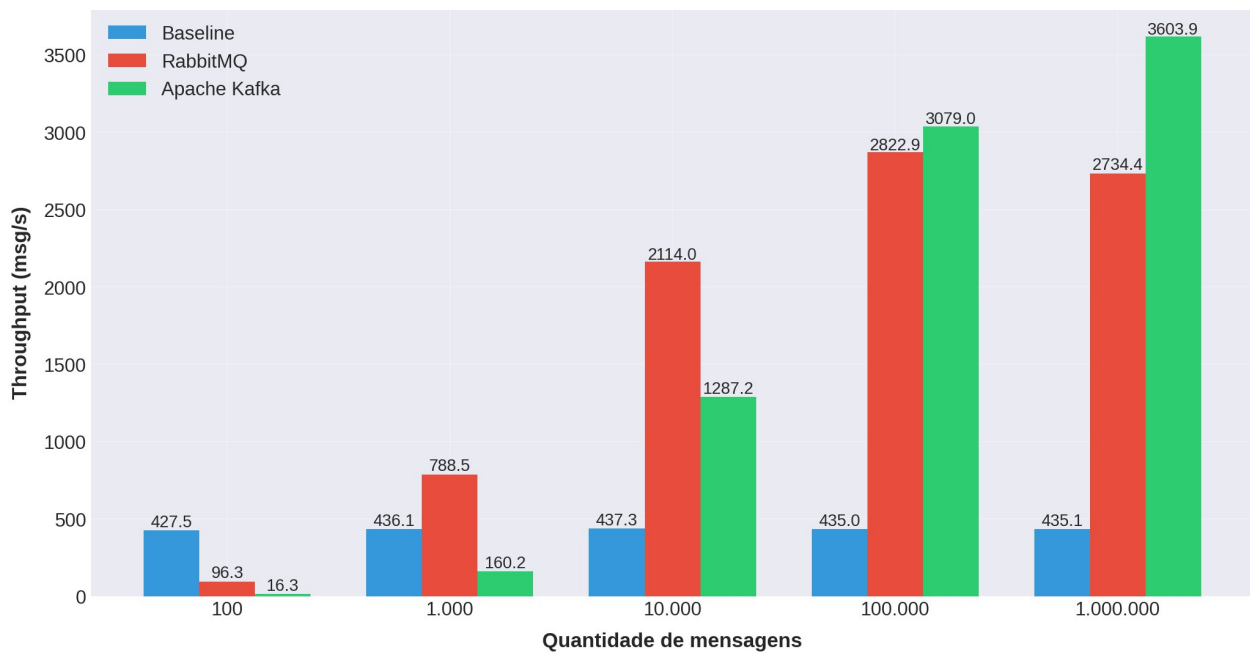
Conforme discutido em mais detalhes na seção 2.5.2, a vazão, de maneira geral, representa a capacidade máxima de trabalho útil que o sistema consegue realizar por unidade de tempo. No contexto deste estudo, essa métrica reflete quantas mensagens a arquitetura consegue persistir e, efetivamente, entregar. Tendo isso em mente, as Figuras 5 e 6 isolam os dados de vazão para permitir uma análise detalhada do comportamento de cada tecnologia nos tamanhos de mensagem de 100 e 1.000 bytes, respectivamente.

Figura 5 – Comparativo de Vazão para mensagens de 100 bytes



Fonte: Elaboração própria (2025)

Figura 6 – Comparativo de Vazão para mensagens de 1.000 bytes



Fonte: Elaboração própria (2025)

Durante a execução controlada dos 30 ciclos de teste para cada cenário, não foram observados erros de transmissão, perdas de mensagens ou *timeouts*. Embora as configurações de alta disponibilidade (Consenso) estivessem ativas - impondo o *overhead* computacional de replicação - o experimento focou em medir o impacto desse *overhead* no desempenho

(Vazão/Latência) em regime normal de operação, e não a capacidade de recuperação de falhas, o que constituiria um estudo de Engenharia do Caos. Dito isso, a análise desses dados permite identificar três comportamentos arquiteturais distintos: O Teto de Processamento do Modelo Síncrono; A Eficiência do RabbitMQ em Cargas Intermediárias; O Ponto de Inflexão do Apache Kafka.

5.2.1 O Teto de Processamento do Modelo Síncrono

O resultado mais consistente (e revelador) do experimento é o comportamento da implementação *Baseline* (HTTP). Conforme observado nas tabelas, a vazão desse modelo permaneceu estagnada em torno de 430 mensagens por segundo, independentemente do tamanho da mensagem e da quantidade total de mensagens enviadas.

Esse fenômeno corrobora a teoria de Tanenbaum e Van Steen (2017) sobre o custo do bloqueio de Entrada/Saída em comunicações síncronas. No modelo HTTP implementado, cada requisição exige a abertura de uma conexão, o envio do cabeçalho, o processamento sequencial no servidor e o aguardo da resposta.

Essa natureza bloqueante obriga o cliente a "parar e esperar". Dessa forma, mesmo que a rede suporte *gigabits* de largura de banda, a latência de ida e volta somada ao tempo de processamento impõe um limite físico intransponível para uma *thread* única. Newman (2019) alerta que esse acoplamento temporal é a principal causa de gargalos em microserviços, pois a vazão do sistema fica limitada pelo componente mais lento da cadeia, impedindo o aproveitamento real dos recursos de *hardware* disponíveis.

5.2.2 A Eficiência do RabbitMQ em Cargas Intermediárias

O RabbitMQ demonstrou um desempenho superior em cenários de carga leve e média. Nos cenários *Size 2* (1.000 mensagens) e *Size 3* (10.000 mensagens), ele atingiu, tanto em mensagens com tamanho de 100 bytes quando de 1.000 bytes, resultados que superaram o *Baseline* e o Apache Kafka. Esse ganho, de maneira geral, deve-se ao desacoplamento assíncrono. Ou seja, o produtor RabbitMQ, configurado para envio sem confirmação síncrona bloqueante, não precisa aguardar a persistência em disco de cada mensagem individualmente antes de enviar a próxima.

Apesar disso, observa-se que o crescimento da vazão do RabbitMQ desacelera a partir do *Size 4*, com o *Size 5* apresentando, inclusive, desempenho inferior, o que evidencia

indícios de saturação no sistema. Segundo Muratbekov (2024), embora as *Quorum Queues* utilizem o algoritmo Raft para alta disponibilidade, a necessidade de replicar cada mensagem para a maioria dos nós (mesmo em um *cluster* unitário simulando escrita em disco segura) impõe um *overhead* de coordenação e Entrada/Saída que limita a vazão máxima se comparado à abordagem de *log* puro. Em outras palavras, o custo de replicação do algoritmo Raft (usado nas *Quorum Queues*) é sensível ao volume de dados trafegados na rede interna do *cluster* lógico.

5.2.3 O Ponto de Inflexão do Apache Kafka

O comportamento do Apache Kafka ilustra classicamente o *trade-off* entre latência e vazão. Nos cenários iniciais (*Size 1* e *Size 2*), sua vazão foi inferior tanto ao *Baseline* quanto ao RabbitMQ. Isso ocorre devido ao mecanismo de loteamento agressivo onde o Kafka aguarda acumular um volume de dados, ou um tempo limite, antes de realizar o envio (SHAPIRA *et al.*, 2017). Em cargas pequenas, esse tempo de espera domina a métrica, prejudicando o resultado final.

Apesar disso, nota-se de forma evidente que a partir do *Size 4* (100.000 mensagens), ocorre o ponto de inflexão. Neste ponto, o Kafka ultrapassa o RabbitMQ, com uma vantagem sendo ainda mais ampliada no *Size 5* (1.000.000 de mensagens), que é justamente onde a tecnologia alcança o seu pico de vazão, registrando o maior valor em todo o experimento.

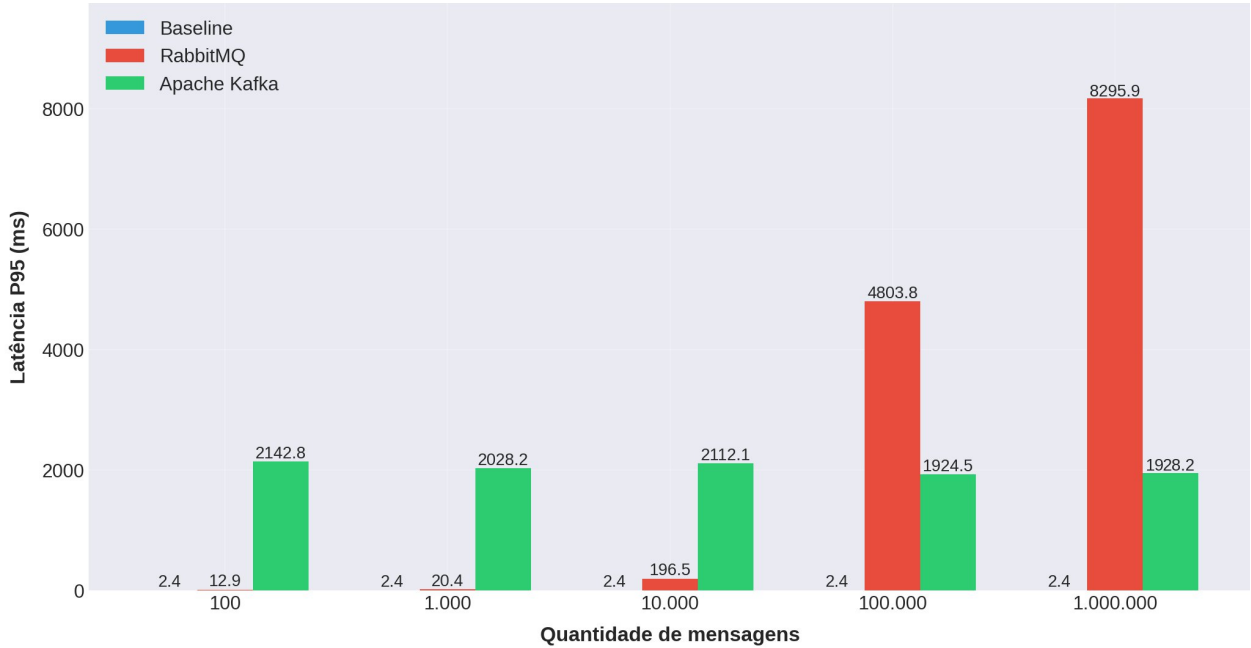
Essa escalabilidade em cargas massivas é explicada pela arquitetura de *log* sequencial. Kreps *et al.* (2011) detalham que o Kafka utiliza operações de escrita linear em disco, que são ordens de magnitude mais rápidas que escritas aleatórias. Além disso, o uso da chamada de sistema *sendfile* (*Zero-Copy*) permite transferir dados do disco para a rede sem copiá-los para o espaço de usuário, maximizando a eficiência da CPU (*Central Processing Unit*) e da memória. Fora isso, o protocolo KRaft, utilizado nesta versão 4.0, contribui para esse resultado ao remover a sobrecarga de comunicação com o ZooKeeper, permitindo que o *broker* gerencie metadados de forma mais ágil.

5.3 ANÁLISE DE LATÊNCIA DA CAUDA (P95 E P99)

A análise de latência em sistemas distribuídos não deve limitar-se à média aritmética, uma vez que esta métrica frequentemente mascara flutuações momentâneas de desempenho. Conforme postulado por Barroso e Dean (2013) no estudo sobre "A Cauda em Escala" (*The*

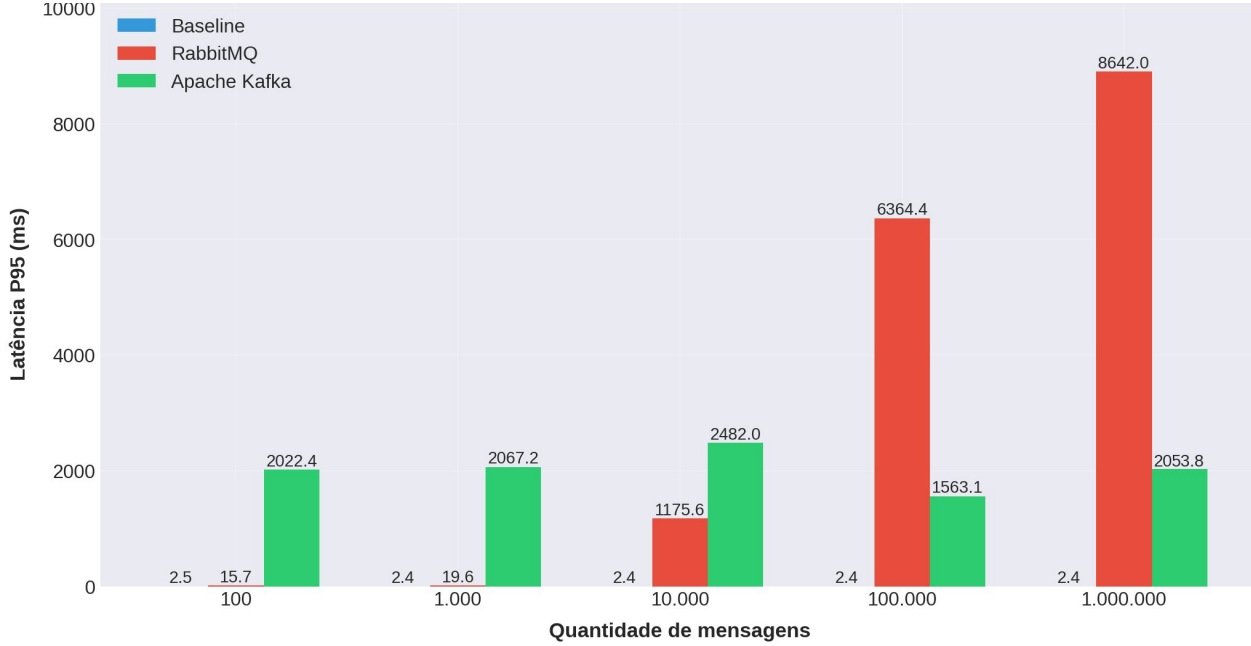
Tail at Scale), a estabilidade de um sistema é melhor representada pelos percentis superiores, especificamente o P95 e o P99. A partir dessas considerações, as Figuras 7 a 10 apresentam, de forma sequencial, o comportamento das métricas analisadas em cada um dos cenários contemplados pelo experimento.

Figura 7 – Comparativo da Latência P95 para mensagens de 100 bytes



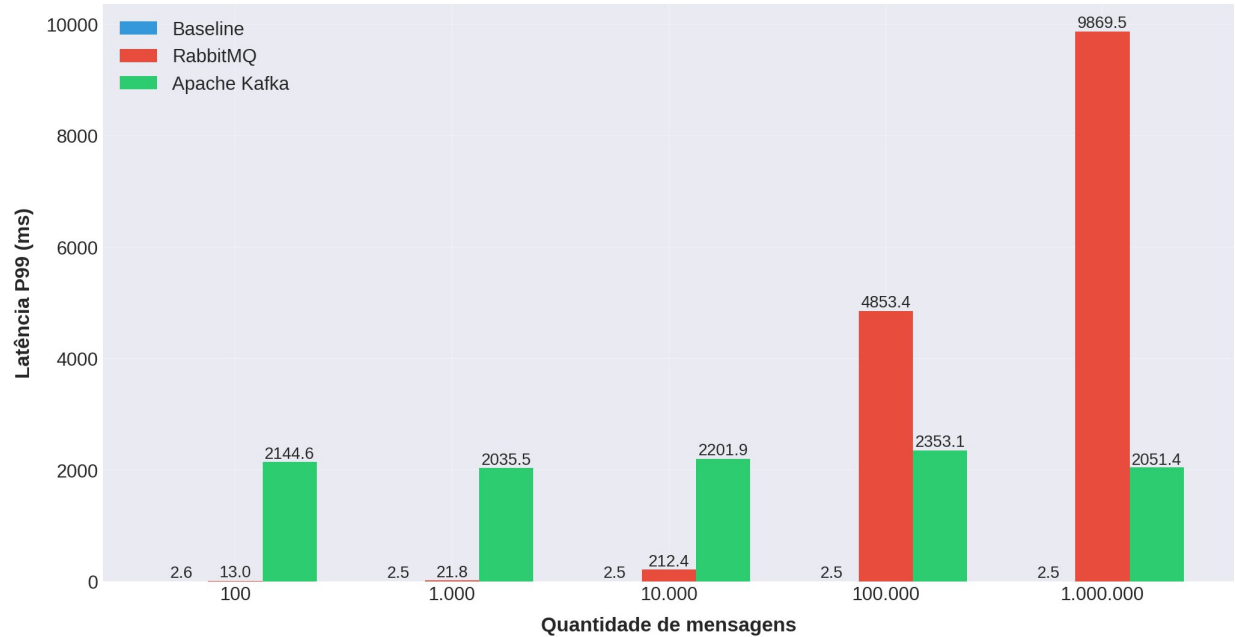
Fonte: Elaboração própria (2025)

Figura 8 – Comparativo da Latência P95 para mensagens de 1.000 bytes



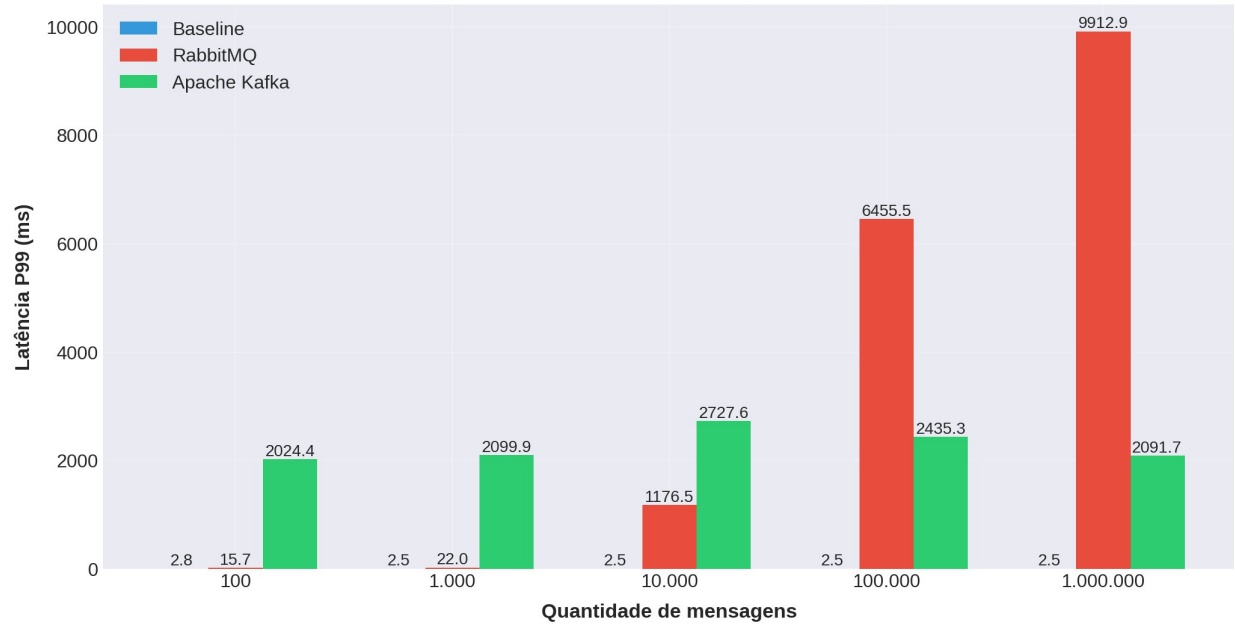
Fonte: Elaboração própria (2025)

Figura 9 – Comparativo da Latência P99 para mensagens de 100 bytes



Fonte: Elaboração própria (2025)

Figura 10 – Comparativo da Latência P99 para mensagens de 1.000 bytes



Fonte: Elaboração própria (2025)

A partir da análise dos dados apresentados nas Figuras 7 a 10, é possível identificar distinções arquiteturais significativas entre os modelos, as quais serão detalhadas nas subseções subsequentes.

5.3.1 O Custo da *Batching* no Apache Kafka

Um dos resultados mais contra-intuitivos observados ocorre nos cenários de microescala (*Size 1* e *Size 2*). Enquanto o *Baseline* e o RabbitMQ mantêm latências na ordem de milissegundos, o Apache Kafka exibe uma latência P99 superior a 2.000 ms (2 segundos).

Este comportamento não indica uma falha, mas sim uma característica intrínseca de configuração padrão voltada para a vazão. Conforme descrito por Shapira *et al.* (2017), o produtor Kafka utiliza um mecanismo de acumulação de mensagens (*batching*) para reduzir o número de chamadas de Entrada/Saída. Em outras palavras, o produtor é instruído a aguardar a formação de um lote ou o expirar de um temporizador antes de transmitir os dados.

Em cenários com poucas mensagens (100 ou 1.000), o *buffer* de envio demora a ser preenchido, forçando o sistema a aguardar o tempo limite, o que infla artificialmente a latência da primeira mensagem. Esse resultado valida empiricamente que o Kafka, em sua configuração padrão, prioriza a eficiência de rede e disco em detrimento da latência de eventos individuais esporádicos.

5.3.2 A Estabilidade das *Quorum Queues* no RabbitMQ

O RabbitMQ demonstrou uma grande consistência na latência entre todos os cenários testados até o ponto de saturação. Mesmo operando com *Quorum Queues*, que exigem confirmação de escrita via algoritmo Raft em disco (MURATBEKOV, 2024), a latência P99 manteve-se com um valor razoável até o cenário *Size 3*.

Essa estabilidade deve-se ao modelo de *push* do protocolo AMQP. Diferentemente do Kafka, que aguarda o consumidor buscar dados (*poll*), o RabbitMQ envia as mensagens assim que elas estão disponíveis e confirmadas, mantendo um fluxo contínuo e de baixa latência para cargas moderadas. O experimento confirma a tese de Souza (2020) de que o RabbitMQ oferece tempos de resposta superiores para aplicações transacionais onde o volume de dados não atinge a escala de *Big Data*.

5.3.3 Degradação em Escala Massiva

No cenário de carga máxima (*Size 5*, com 1.000.000 de mensagens), observa-se uma estabilidade notável no fenômeno da latência de cauda do Apache Kafka, mantendo-se na casa dos 2.000 ms (2 segundos) tanto para 100 bytes quanto para 1.000 bytes. Embora 2

segundos seja uma latência considerada alta para padrões de tempo real, o fato dela não ter degradado em relação aos cenários menores (*Size 1* ou *2*) comprova que a arquitetura do Kafka não sofre com o aumento do volume de dados da mesma forma que o RabbitMQ. Muito pelo contrário, o Kafka escala linearmente, enquanto o RabbitMQ encontrou um teto operacional severo no cenário de teste massivo.

A latência de cauda observada no Kafka é característica de sistemas baseados na JVM (*Java Virtual Machine*) sob pressão de memória. Conforme Kreps *et al.* (2011), o Kafka depende fortemente do *Page Cache* do sistema operacional para otimizar operações de leitura e escrita. Em cargas massivas, a concorrência por operações de Entrada/Saída (E/S) de disco – entre a escrita de novos segmentos de *log* e a leitura pelo consumidor – somada a eventuais pausas para coleta de lixo (*Garbage Collection*), pode ocasionar bloqueios momentâneos que elevam a latência de requisições específicas.

Esse comportamento corrobora as observações de Barroso e Dean (2013), segundo os quais sistemas em larga escala tendem a apresentar excelente desempenho médio, porém maior variabilidade (*jitter*⁹). Dessa forma, embora a vazão global permaneça alta, uma pequena porcentagem de requisições sofre atrasos significativos devido à contenção de recursos compartilhados.

5.4 SÍNTESE COMPARATIVA E DIRETRIZES ARQUITETURAIS

A seleção de um *middleware* de mensageria não deve ser pautada apenas por métricas isoladas de vazão máxima, mas sim pela adequação da tecnologia aos requisitos de nível de serviço da aplicação. Em outras palavras, os experimentos demonstraram que não existe uma "bala de prata": cada solução apresenta uma zona de eficiência ótima e outra zona de degradação.

Para aplicações de pequeno porte ou microsserviços com tráfego esporádico (teoricamente representados pelos cenários *Size 1* e *Size 2*), a introdução de um *broker* adiciona complexidade acidental desnecessária. Sendo assim, os dados indicam que o *Baseline* síncrono oferece a menor latência absoluta.

Neste contexto, o custo de rede para o *handshake*¹⁰ com o *broker*, somado à persistência em disco, supera o tempo de processamento da própria mensagem. Portanto, para

⁹ *Jitter* refere-se à variação na latência de entrega dos pacotes ou mensagens. Em sistemas de tempo real, um *jitter* elevado é indesejável pois torna o tempo de processamento imprevisível (KUROSE; ROSS, 2021).

¹⁰ Termo que se refere ao processo de estabelecimento de conexão entre o cliente e o *broker*.

sistemas onde a vazão não exceda, por exemplo, 400 requisições por segundo e a durabilidade extrema não seja mandatória, a arquitetura REST (*Representational State Transfer*) tradicional permanece a escolha mais eficiente em termos de recursos e simplicidade operacional (NEWMAN, 2019).

Por outro lado, o RabbitMQ, configurado com *Quorum Queues*, consolidou-se como a solução mais robusta para cenários transacionais de médio porte. Sua capacidade de manter a latência P99 baixa, mesmo sob carga considerável, o torna ideal para sistemas onde a "latência de cauda" impacta diretamente o usuário final, como processamento de pagamentos ou atualizações de inventário em tempo real.

A vantagem arquitetural reside no modelo de *push* e na confirmação individual. Conforme destacado por Muratbekov (2024), a garantia de consistência forte (CP) provida pelo algoritmo Raft nas *Quorum Queues* assegura que a mensagem não seja perdida, sem impor o tempo de espera característico do processamento em lote.

Já em relação ao Apache Kafka, conclui-se claramente que sua hegemonia é exclusiva em cenários de alto volume de dados. O "ponto de inflexão" observado entre o *Size 4* e o *Size 5* demonstra que a arquitetura de *log* sequencial brilha quando o objetivo é maximizar a vazão de ingestão, sacrificando um pouco da latência individual em prol da eficiência coletiva.

6 CONCLUSÃO

A transição de arquiteturas monolíticas para sistemas distribuídos baseados em microsserviços impôs desafios significativos à engenharia de *software*, tornando a eficiência da comunicação entre processos um fator determinante para o sucesso de uma aplicação. Este trabalho propôs-se a investigar, através de uma abordagem experimental, como as versões mais recentes dos *message brokers* líderes de mercado – RabbitMQ e Apache Kafka – comportam-se frente às demandas de escalabilidade e latência, contrastando-os com o modelo síncrono tradicional.

A análise empreendida permitiu ultrapassar o senso comum de que "Kafka é rápido" e "RabbitMQ é confiável", revelando uma realidade mais nuançada governada por decisões de *design* e *trade-offs* de algoritmos de consenso. O objetivo geral deste trabalho, de realizar uma análise comparativa de desempenho e escalabilidade entre RabbitMQ e Apache Kafka, foi plenamente atingido através da execução dos objetivos específicos. Quanto à implementação do ambiente experimental, o estudo entregou uma arquitetura reproduzível baseada em *containers* Docker, isolando as variáveis de ambiente. A adoção das tecnologias mais recentes (RabbitMQ 4.1.1 com *Quorum Queues* e Apache Kafka 4.0 com KRaft) provou-se fundamental. Observou-se que a remoção do ZooKeeper no Kafka simplificou a infraestrutura, enquanto o algoritmo Raft no RabbitMQ garantiu consistência forte sem degradar a latência em níveis proibitivos, preenchendo a lacuna identificada na revisão bibliográfica sobre a falta de estudos com essas versões modernas.

No que tange à execução dos perfis de carga, a submissão dos sistemas a cinco cenários progressivos (de 100 a 1.000.000 de mensagens) e em diferentes tamanhos de mensagens enviadas (100 bytes e 1.000 bytes) revelou os limites físicos de cada abordagem. O *Baseline* síncrono atingiu seu teto de saturação precocemente, estagnando em aproximadamente 430 mensagens por segundo devido ao bloqueio de Entrada/Saída, confirmando as previsões teóricas de Tanenbaum e Van Steen (2017) sobre a ineficiência do acoplamento temporal em escala.

Referente à mensuração de métricas, a análise de latência de cauda (P95 e P99) trouxe os *insights* mais valiosos do estudo. Ficou evidente que o mecanismo de *batching* do Apache Kafka, embora essencial para sua alta vazão final, impõe um custo severo de latência em baixas cargas (superando 2 segundos), tornando-o inadequado para interações em tempo real com baixo volume. Em contrapartida, o RabbitMQ demonstrou ser a solução mais

equilibrada para a maioria dos casos de uso empresariais (Médio Porte), mantendo latência P99 baixa e vazão alta antes da saturação.

Por fim, no que diz respeito às diretrizes de seleção, o estudo consolidou que a escolha não é binária. Para aplicações de pequeno porte, o custo operacional de um *broker* não se justifica frente à baixa latência do HTTP. Para sistemas transacionais críticos que exigem consistência imediata e baixa latência individual, o RabbitMQ é a escolha superior. Já para cenários de *Big Data*, onde a vazão agregada supera a importância da latência unitária, o Apache Kafka é imbatível.

À vista disso, este trabalho contribui para a comunidade acadêmica e industrial ao fornecer evidências de que as novas arquiteturas de consenso (Raft e KRaft) alteraram alguns comportamentos padrões dessas ferramentas. Demonstrou-se que o "medo" histórico de perda de dados no RabbitMQ é mitigado pelas *Quorum Queues* com um impacto de desempenho aceitável para a maioria das aplicações comerciais. Simultaneamente, desmistificou-se a ideia de que o Kafka deve ser usado para tudo; seu *overhead* de latência em microescala foi quantificado, servindo de alerta para arquitetos que consideram sua adoção em cenários de baixa volumetria apenas por "hype" tecnológico. A disponibilização do código-fonte aberto e da metodologia de instrumentação permite que outros pesquisadores repliquem e estendam esses testes.

Sendo assim, com base nos resultados obtidos e nas limitações identificadas, sugerem-se as seguintes linhas de investigação para trabalhos futuros:

- Engenharia do Caos: aplicar metodologias de *Chaos Engineering* para avaliar a resiliência dos sistemas e a potencial perda de dados durante falhas abruptas de infraestrutura, além de reexecutar cada cenário múltiplas vezes, o que permitiria um cálculo mais assertivo do desvio padrão e do intervalo de confiança, que também reduz o impacto de variações pontuais do ambiente..
- Comparação com tecnologias emergentes: estender o *benchmark* para incluir soluções como Apache Pulsar e NATS JetStream que, apesar de sua recente inserção no mercado, propõem-se a combinar a baixa latência característica do RabbitMQ com a alta vazão do Kafka.
- Análise de eficiência energética: avaliar o consumo de recursos de *hardware* (CPU, memória e energia) por mensagem processada, métrica cada vez mais relevante no contexto de *Green Computing*.

- Investigação de outros parâmetros: sugere-se também a investigação de parâmetros não abordados neste escopo, como variação do número de produtores e consumidores, por exemplo.

Em síntese, conclui-se que tanto o RabbitMQ quanto o Apache Kafka amadureceram significativamente em suas versões recentes. A escolha fundamentada entre essas tecnologias, orientada pelos dados aqui apresentados, constitui etapa primordial para a construção de sistemas distribuídos resilientes e eficientes.

REFERÊNCIAS

- ANDREASSEN, R. *et al.* Upgrade of the CERN RADE framework architecture using RabbitMQ and MQTT. *In: INTERNATIONAL CONFERENCE ON ACCELERATOR AND LARGE EXPERIMENTAL PHYSICS CONTROL SYSTEMS*, 16., 2017, Barcelona. **Anais [...]**. Barcelona: JACoW, 2017. p. 1390-1395.
- ARRUDA, F.; MARTINS, D. **Análise Comparativa entre Sistemas de Mensageria: Apache Kafka vs. RabbitMQ**. 2020. Trabalho de Conclusão de Curso - Instituto Federal do Sudeste de Minas Gerais, 2020.
- BARROSO, L.; DEAN, J. The tail at scale. **Communications of the ACM**, New York, v. 56, n. 2, p. 74-80, 2013.
- BMW GROUP. **Revving Up IoT Use Cases with Real-Time Hybrid Cloud Data Architecture**. BMW Tech Blog, 12 ago. 2021. Disponível em: <https://www.confluent.io/customers/bmw-group/>. Acesso em: 10 maio 2025.
- BOSCHI, S.; SANTOMAGGIO, G. **RabbitMQ Cookbook**. Birmingham: Packt, 2013.
- CLOUDAMQP. **Part 1: RabbitMQ for beginners - What is RabbitMQ?** CloudAMQP Blog, 22 out. 2025. Disponível em: <https://www.cloudamqp.com/blog/part1-rabbitmq-for-beginners-what-is-rabbitmq.html>. Acesso em: 5 nov. 2025.
- CLOUDKARAFKA. **Part 1: Apache Kafka for beginners - What is Apache Kafka?** CloudKarafka Blog, 19 mar. 2020. Disponível em: <https://www.cloudkarafka.com/blog/part1-kafka-for-beginners-what-is-apache-kafka.html>. Acesso em: 5 nov. 2025.
- COULOURIS, G. *et al.* **Distributed systems: concepts and design**. 6. ed. Harlow: Pearson, 2021.
- DeCANDIA, G. *et al.* Dynamo: Amazon's highly available key-value store. *In: SYMPOSIUM ON OPERATING SYSTEMS PRINCIPLES*, 21., 2007, Stevenson. **Proceedings [...]**. New York: ACM, 2007. p. 205-220.
- DOBBELAERE, P.; ESMAILI, K. **Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations**. *In: Distributed Event-Based Systems (DEBS)*. ACM, 2017. p. 227–238.
- DUNNE, J.; GRACIOLI, G.; FISCHMEISTER, S. A comparison of data-streaming frameworks for anomaly detection in embedded systems. *In: REAL-TIME SYSTEMS SYMPOSIUM*, 2018, Porto. **Proceedings [...]**. Los Alamitos: IEEE, 2018.
- EUGSTER, P. T. *et al.* The many faces of publish/subscribe. **ACM Computing Surveys**, New York, v. 35, n. 2, p. 114-131, 2003.
- FU, G.; ZHANG, Y.; YU, G. A fair comparison of message queuing systems. **IEEE Access**, [s. l.], v. 9, p. 98423-98438, 2021.

GHEMAWAT, S.; GOBIOFF, H.; LEUNG, S.-T. The Google file system. *In: SYMPOSIUM ON OPERATING SYSTEMS PRINCIPLES*, 19., 2003, Bolton Landing. **Proceedings** [...]. New York: ACM, 2003. p. 29-43.

GILBERT, S.; LYNCH, N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. **SIGACT News**, New York, v. 33, n. 2, p. 51-59, 2002.

HYNDMAN, R. J.; FAN, Y. Sample quantiles in statistical packages. **The American Statistician**, [s. l.], v. 50, n. 4, p. 361-365, nov. 1996.

JAIN, R. **The art of computer systems performance analysis**. New York: Wiley, 1991.

KREPS, J.; NARKHEDE, N.; RAO, J. Kafka: a distributed messaging system for log processing. *In: NETDB*, 2011, Athens. **Proceedings** [...]. New York: ACM, 2011.

KUROSE, J. F.; ROSS, K. W. **Computer Networking: A Top-Down Approach**. 8. ed. Pearson, 2021.

MILOŠAVLJEVIĆ, M. *et al.* Comparison of message queue technologies for highly available microservices in IoT. *In: ETRAN*, 65., 2021, Čačak. **Anais** [...]. Belgrado: ETRAN, 2021.

MURATBEKOV, Y. N. Identifying the suitable program for queue management through analysis: Apache Kafka or RabbitMQ. **Вестник науки**, v. 5, n. 74, t. 2, maio 2024.

SHAPIRA, G. *et al.* **Kafka: the definitive guide**. Sebastopol: O'Reilly, 2017.

NEWMAN, S. **Building microservices**. 2. ed. Sebastopol: O'Reilly, 2019.

NIKHIL, A.; CHANDAR, V. **Benchmarking Apache Pulsar, Kafka, and RabbitMQ**. Confluent, 21 ago. 2020. Disponível em: <https://www.confluent.io/blog/kafka-fastest-messaging-system/>. Acesso em: 24 nov. 2025.

GONÇALVES, M. **Event-streaming: Apache Kafka + Kafka Streams**. Medium, 20 jan. 2020. Disponível em: <https://medium.com/@marcelomg21/event-streaming-apache-kafka-kafka-streams-em-uma-arquitetura-de-micro-servi%C3%A7os-136c28d7c9c8>. Acesso em: 17 dez 2025.

ONGARO, D.; OUSTERHOUT, J. In search of an understandable consensus algorithm. *In: USENIX ATC*, 2014, Philadelphia. **Proceedings** [...]. Berkeley: USENIX, 2014. p. 305-319.

RAJE, S. **Performance comparison of message queue methods**. 2019. Dissertação (Mestrado) - University of Nevada, Las Vegas, 2019.

SHARVARI, T.; NAG, G. A study on Modern Messaging Systems - Kafka, RabbitMQ and NATS Streaming. **International Journal of Computer Applications**, [s. l.], v. 178, n. 6, p. 1-6, 2019.

SILBERSCHATZ, A.; GALVIN, P.; GAGNE, G. **Operating system concepts**. 10. ed. Hoboken: Wiley, 2018.

SOUZA, R. de A. **Performance analysis between Apache Kafka and RabbitMQ**. 2020. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) - Universidade Federal de Campina Grande, Campina Grande, 2020.

STACK OVERFLOW. **2024 Developer Survey**. Disponível em: <https://survey.stackoverflow.co/2024/technology>. Acesso em: 16 dez 2025.

TANENBAUM, A.; VAN STEEN, M. **Distributed systems: principles and paradigms**. 3. ed. Upper Saddle River: Pearson, 2017.

VIDELA, A.; WILLIAMS, J. **RabbitMQ in action: distributed messaging for everyone**. Shelter Island: Manning, 2012.

VOGELS, W. Eventually consistent. **Communications of the ACM**, New York, v. 52, n. 1, p. 40-44, 2009.

YANG, Y. **Presto® on Apache Kafka® At Uber Scale**. Uber Engineering Blog, 14 abr. 2022. Disponível em: <https://www.uber.com/en-BR/blog/presto-on-apache-kafka-at-uber-scale>. Acesso em: 30 maio 2025.